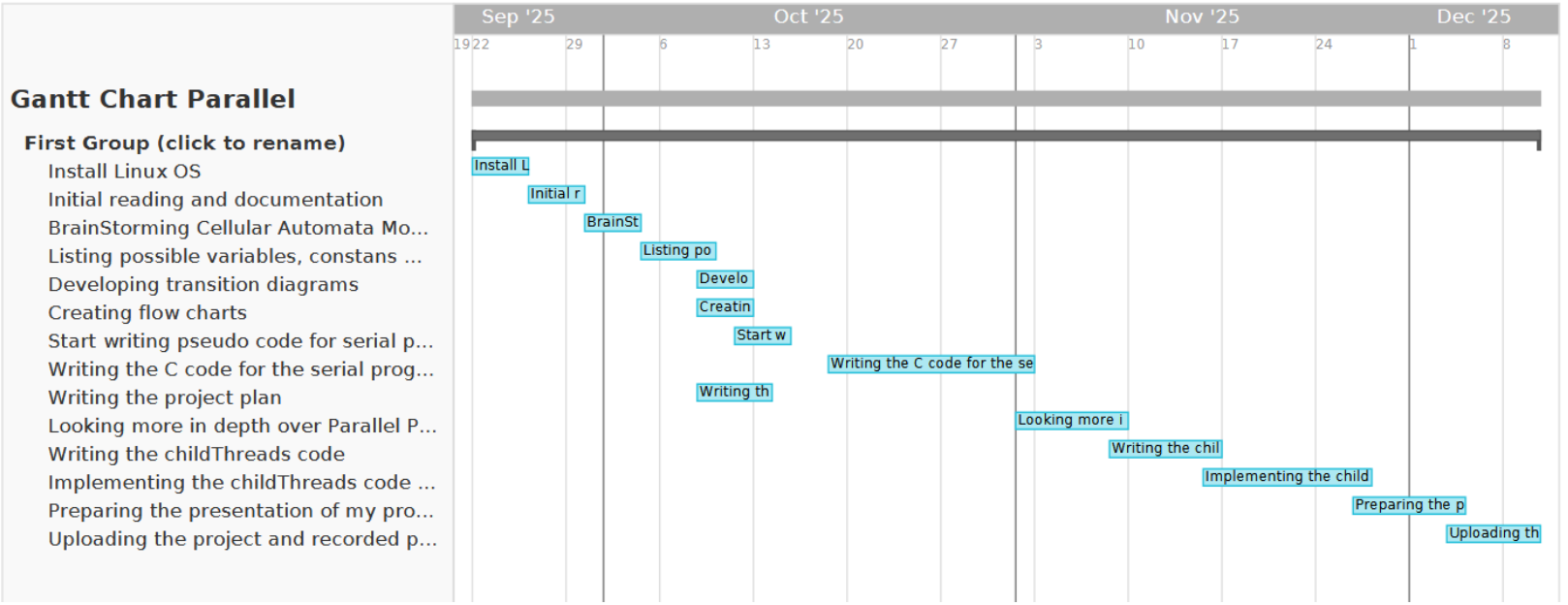


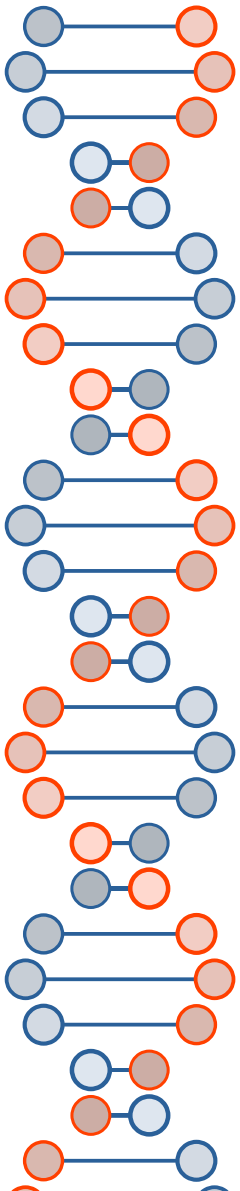
Parallel Programming

Simulating the Spread of the Ebola Virus using Cellular Automata in a Parallel Computing Environment

Kevin Ceausu - B00128171

Project Plan



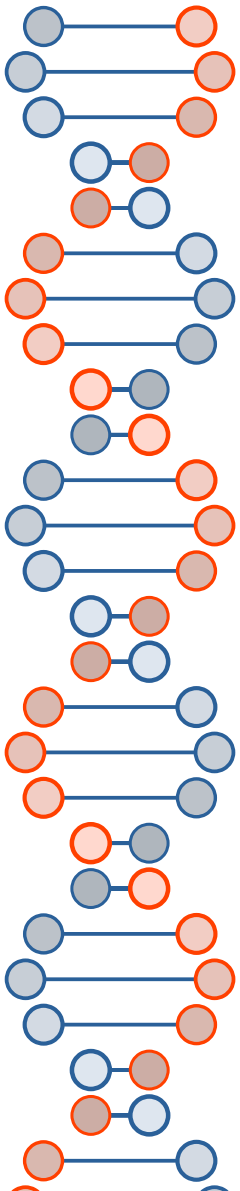


- 1. Installing Linux OS

  This subject along with its project isn’t designed to only teach us about parallel programming, but also to teach us how to adapt and work on a different operating systems instead of only being used to work on Windows. This is why, for this projects development we will be required to use Linux, so we first need to install it.

- 2. Initial Reading and Documentation

  After successfully installing Linux on our personal computer, our first step for this project is to start reading the Project Description and Guidance document along with the paper “A Cellular Automaton Framework for Infectious Disease Spread Simulation” by Pfeifer et al. This will help us develop a more profound idea about what we need to do, what to achieve and how to do it with our project.

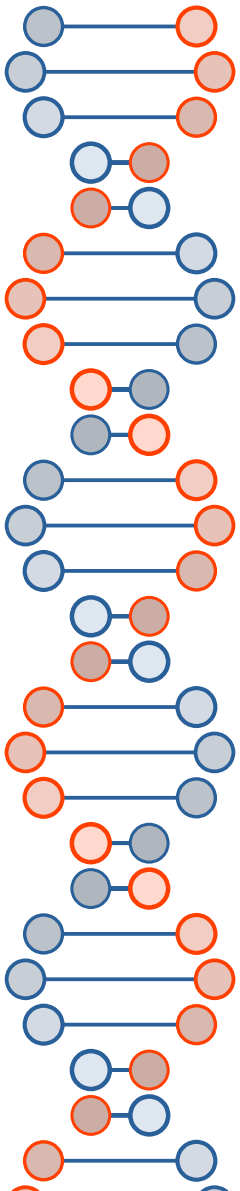


- 3. Brainstorming the Cellular Automata Model

  After doing our initial documentation, we need to start deciding on our approach to the Cellular Automata Model. First we need to get a better understanding of SIR/D basic model and the SEIR/D latent model. We will need to brainstorm ideas about how we will start writing our pseudo-code, actual code, and any charts that come with them.

- 4. Listing possible variables, constants and macros

  In this phase, we are still brainstorming about what exactly we need to have and do with our project, but this time we are doing it in more depth. We are beginning to understand the basics of the Cellular Automata Model and start listing possible variables, constants and macros that we may use in our code.

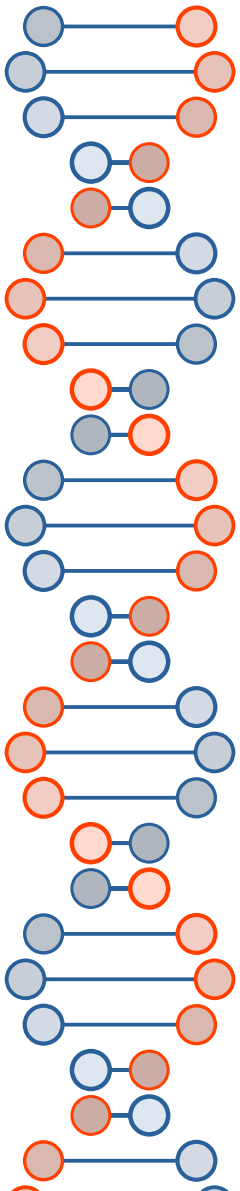


- 5. Developing transition diagrams

  In this phase we are developing transition diagrams of the basic and latent model to develop a better idea of all transitions we will need to implement in our project. Those diagrams will help us create a better image of how the project should work.

- 6. Creating flow charts

  The flow charts will be created in order to help us further plan our development of our code and project, and create a better idea of how exactly we will need to implement variables, constants and macros we previously discussed about.

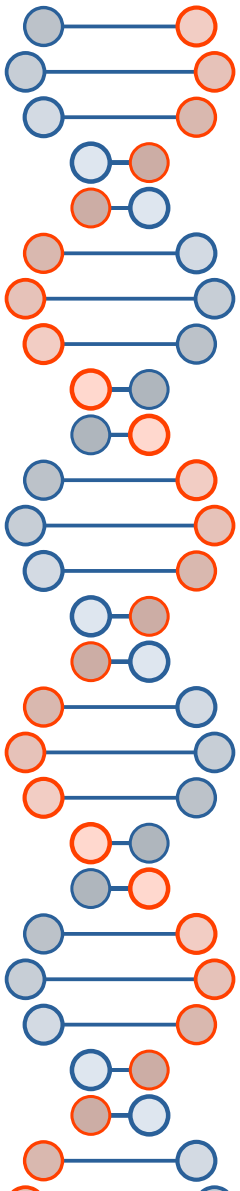


- 7. Start writing pseudo code for serial program

  In order for us to be able to develop our project from scratch and write our actual code, we will first need to write the pseudo code so we can provide a clear description of that steps that will be taken in the process of creating the project.

- 8. Writing the project plan

  Writing the project plan is an important step of developing any project as it’s the tool used to provide us with a clear roadmap for how our project will be executed and controlled, ensuring we don’t fall behind any of our steps and manage to deliver everything on time.

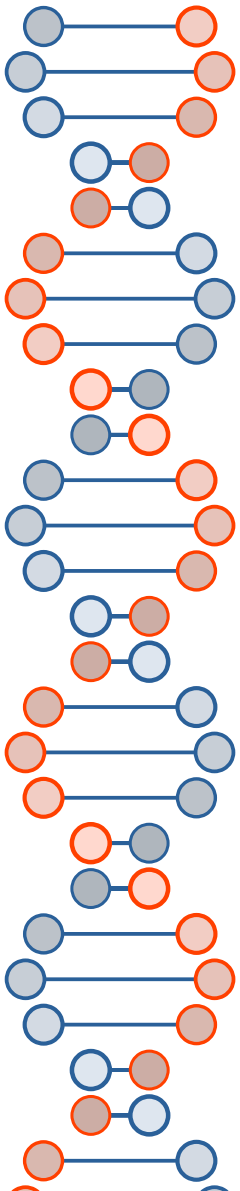


- 9. Writing the C code for the serial program

  This is the part where we actually start developing our project, using the C programming language to write the code in order to have the serial program.

- 10. Looking more in depth over Parallel Programming and Pthreads

  As this is our first time working with Pthreads, we will be required to do more research on how exactly Pthreads work and how they are used in different tasks. By doing this research we will ensure that we have the necessary understanding in order to begin developing our Parallel Program.

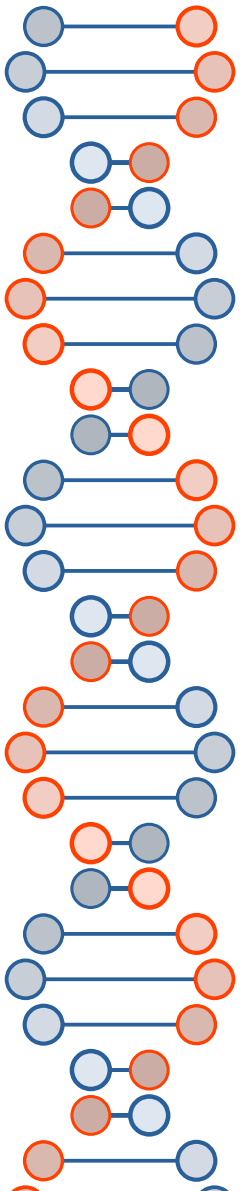


- 11. Writing the Pthreads code

  After doing our research, we will start writing our childThreads code that will allow us to use multithreading for our program.

- 12. Implementing the Pthreads code into my serial program to make it parallel

  With the previous step finished, we can begin working on our parallel program by implementing the Pthreads we have previously worked on to our serial program. This should help our program execute at a faster speed.



- 13. Preparing the presentation of my project

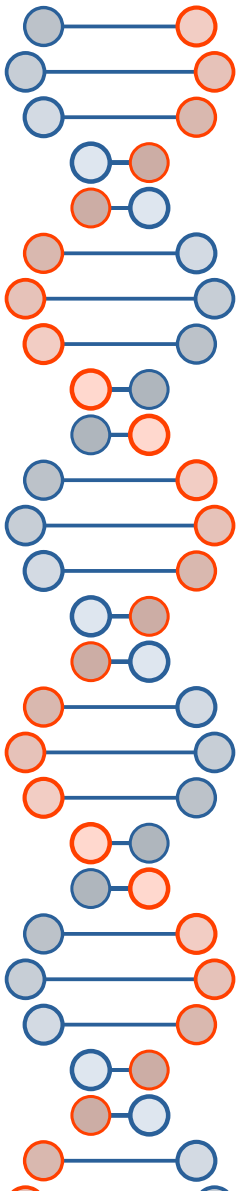
  After managing to conclude my development of the parallel program, I can begin working on the presentation of my project, as all students will need to have a recorded presentation to their project explaining what it does and how it works.

- 14. Uploading the project and recorded presentation

  This is the last step of the project, where everything we have discussed in our project plan has been achieved, and the final program is ready to be uploaded and graded.

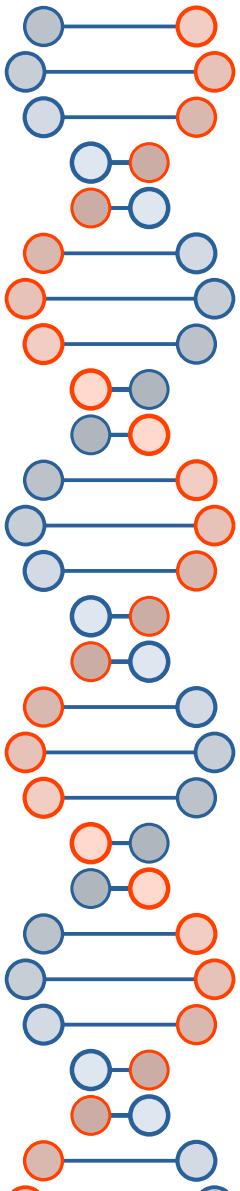
Background/Brief Literature Review

- Ebola virus disease is a severe infection that is transmitted through direct contact with infected bodies.
- Public health responses put an accent on the importance of safe and dignified burials as traditional burials are proven to contribute to further transmission.
- Modelling explores how reducing contact or isolating infectious cases can reduce and affect the epidemic trajectory.
- WHO (2025;2014)



Compartment Models SIR-SEIRD

- Kermack and McKendrick (1927) introduced us to epidemic modelling using compartment models like SIR.
- SEIR model extends SIR by adding the exposed state which represents an incubation period where individuals are infected but not yet infectious.
- Kermack & McKendrick (1927); Tolles & Luong (2020); SEIRS model overview (Nature Methods, 2020)

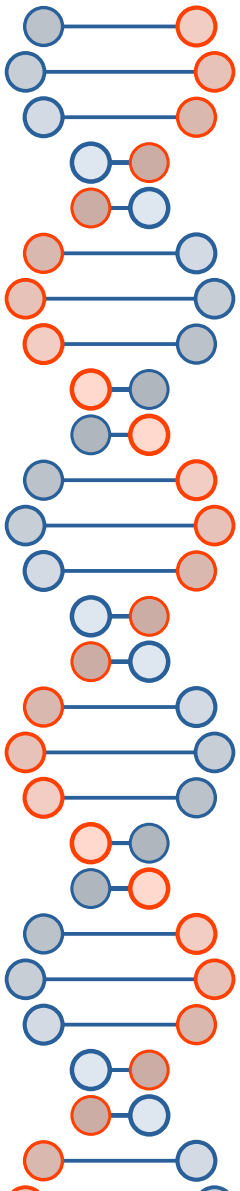


Cellular Automata for epidemics

- Real disease transmission depends on local contact structure.
- Cellular automata captures spatial dynamics by modelling cells on a grid with local neighbour interactions, giving us a realistic simulation of an infectious disease spread.
- CA epidemic models commonly use Moore neighbourhood system to represent local contact around each cell.
- White et al. (2006); Cissé et al. (2013); López et al. (2013)

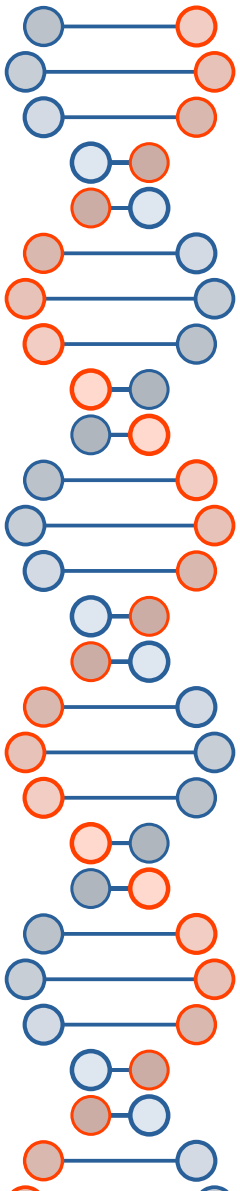
Full references

- Kermack WO, McKendrick AG. A contribution to the mathematical theory of epidemics. Proc. R. Soc. A. 1927.
- Tolles J, Luong T. Modeling epidemics with compartmental models. JAMA. 2020.
- White SH, del Rey AM, Sánchez GR. Modeling epidemics using cellular automata. Appl. Math. Comput. 2006.
- Cissé B, et al. Impact of neighborhood structure on epidemic spreading… Procedia Computer Science. 2013.
- WHO. Ebola disease – Fact sheet. (Updated 24 Apr 2025).
- New WHO safe and dignified burial protocol - key to reducing Ebola transmission (2014)
- CDC, Improving Burial Practices and Cemetery Management During an Ebola Virus Disease Epidemic — Sierra Leone, 2014
- Victory KR, et al. Ebola Transmission Linked to a Single Traditional Funeral Ceremony — Kissidougou, Guinea, December, 2014–January 2015



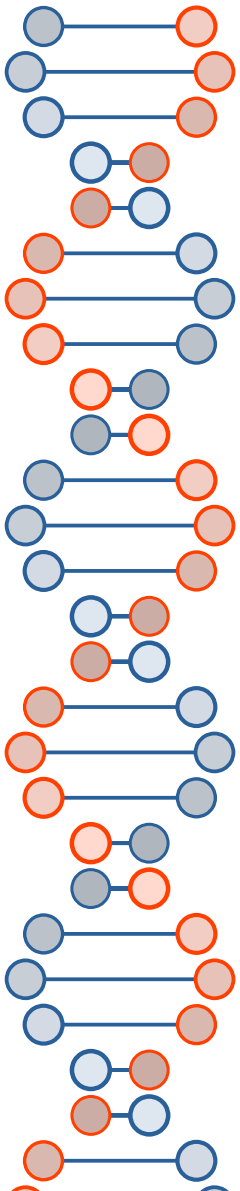
Detailed description of your Cellular Automata Model Rules

- I will be using a 2D grid with the size of rows x columns.
- Each cell represents one individual.
- Moore neighbourhood (8 surrounding cells) will be used.
- Period boundary conditions (“wrap-around”) will be used in order for all cells to have the same amount of neighbours.



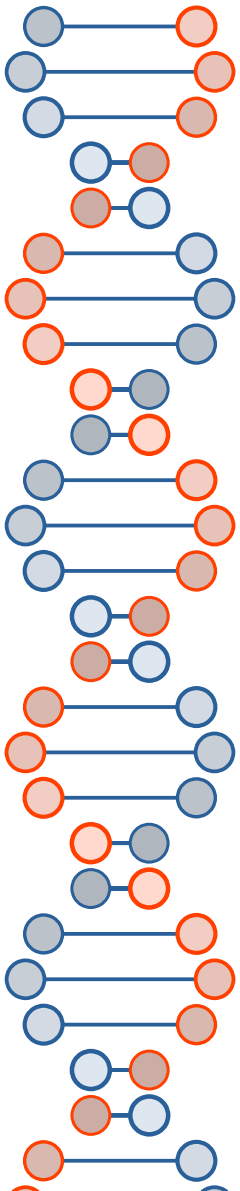
States

- S – Susceptible
- E – Exposed
- I – Infectious
- R – Recovered
- D – Dead
- . – Empty



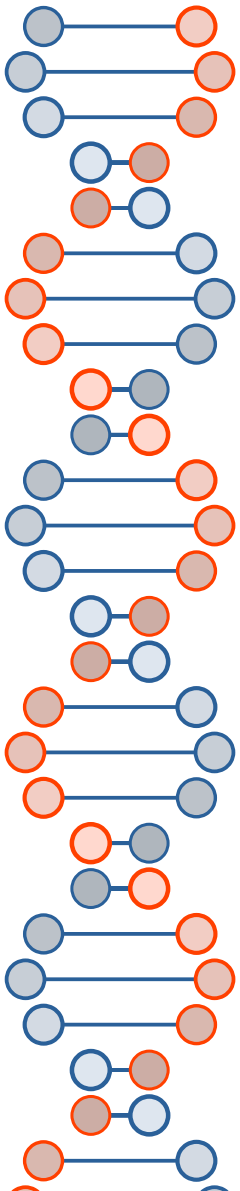
Basic Transitions S to E, E to I

- Infectious is local, based on the numbers of infectious neighbours.
- E has a counter from E to I, when counter reaches 0, cell becomes I.



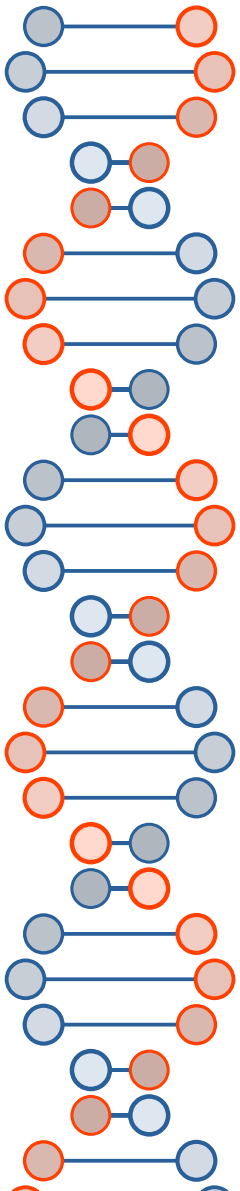
Infectious outcomes, I to R or I to D

- Infectious cells have two possible outcomes, Recover or Die.
- To decide if a cell will die, we calculate by generating a random number and comparing it to the death probability, if random number < death prob = death.



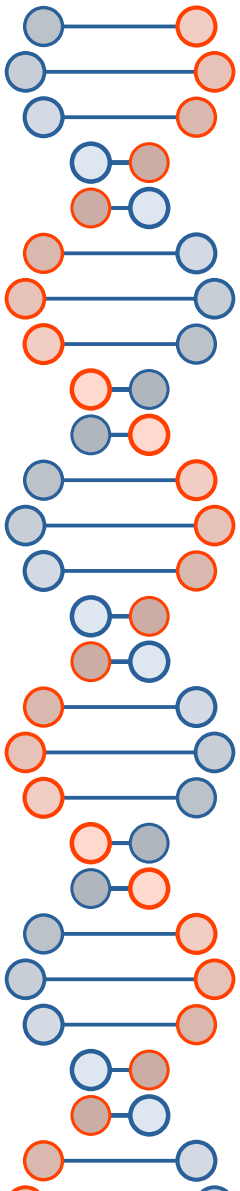
Removal of dead individuals, D to empty

- A counter is assigned to death cells.
- When the counter reaches 0, the cell becomes empty.



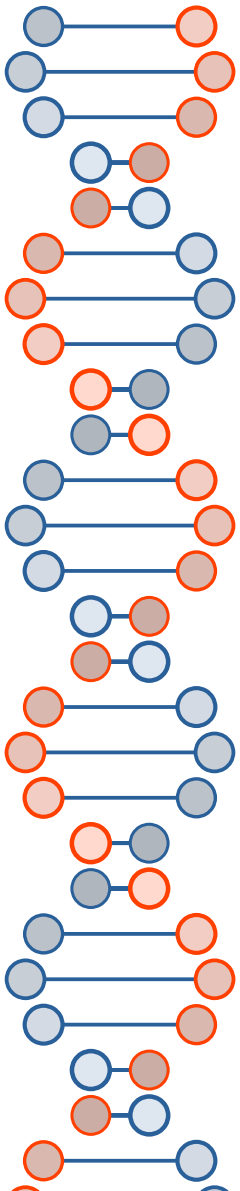
Update scheme

- There are two grids, current used to read, and future used to write.
- All transitions are written into future.
- At the end of the day, we swap current and future.



Pseudo-Code and Flow Charts

- The following slides will be about the development of parallel_basic.c and its child thread and the parallel_latent.c and its child thread.



Pseudo Code for parallel_basic.c

includes standard libraries : stdio, stdlib, time, pthread

timer.h for get_time macro

vars_defs_functions.h for macros, variables and functions control

Global/File variables:

time_range_I_to_R_plusOne

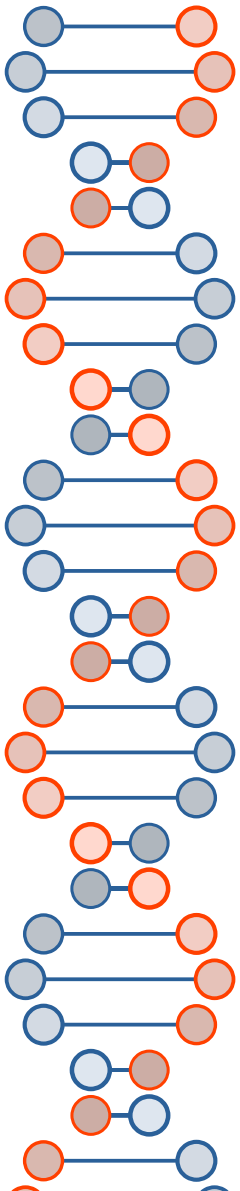
thread_count

barrier// pthread barrier used to synchronise threads each day

current, future

fp_daySIRD// file pointer to write sird data

localS, localI, localR, localD// arrays per thread_count to avoid race conditions



Function prototype:

child_thread(rank) //thread function defined in other file

Function allocate_worlds():

Allocate array of rows pointers for current and future

For each row I :

allocate COLS CELL for current[i]

allocate COLS CELL for future[i]

if malloc fails > print error and exit

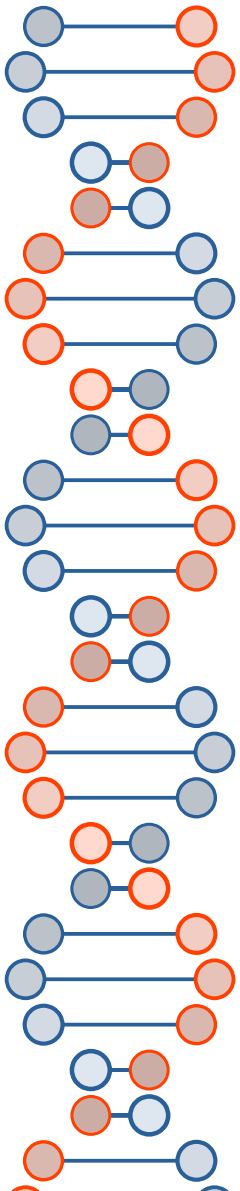
Function free_worlds():

if current exists:

for each row I: free(current[i])

free current and set to NULL

do the same for future



Function main(argc, argv):

1. validate input arguments:

if argc < 2: print usage and exit

thread_count = atoi(argv[1])

if thread_count <= 0: print error and exit

if rows are not divisible by thread_count: print error and exit

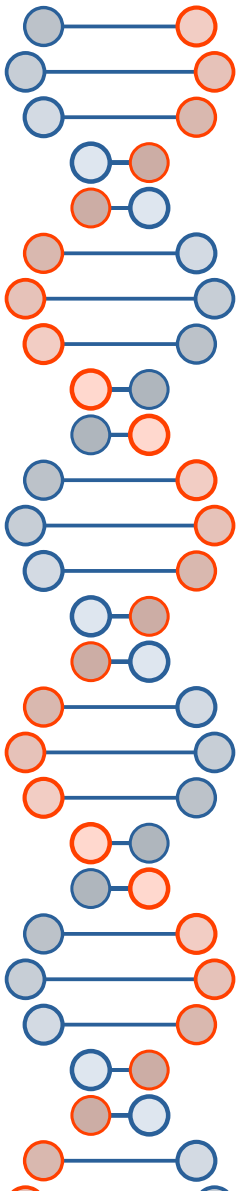
2. initialisation constant:

time range I to R plus one = Max_Time_I_to_R – Min_Time_I_to_R + 1

3. Allocate the two worlds:

allocate_worlds()

4. Set day 0 global counters for numS, numI, numR, numD = 0



5. Open output file data/daySIRD.dat

6. Seed random number generator

srand48(time(null))

7. Initialise world

intialise_world(current, future, numS, numI)

8.Output day 0:

OutputWorld(0, current)

outputdaySIRD(fp_daySIRD, 0, numS, numI, numR, numD)

9. Allocate thread count arrays:

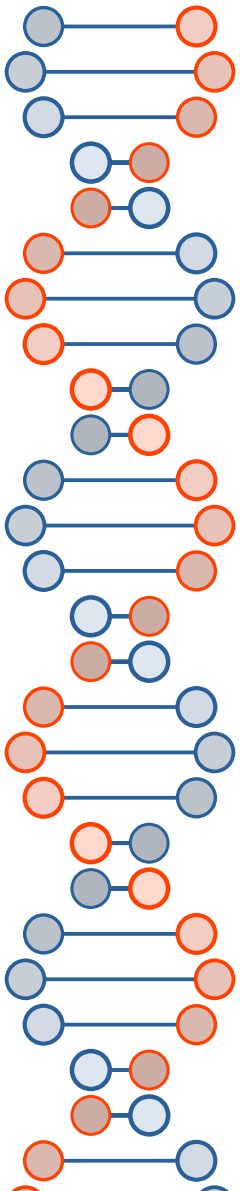
localS, localI, localR, localD = calloc(thread_count)

10. Initialise barrier for thread_count threads:

pthread_barrier_init()

11. Allocate thread handle array:

thread_handles = malloc(thread_count)



12. Start timer:

GET_TIME(START)

13. Create threads

for thread id in 0 >> thread_count -1:

pthread_create(handles[thread], child_thread)

14. Join threads

for thread id in 0 >> thread_count -1:

pthread_join(thread_handles[thread]

15. Stop timer and print:

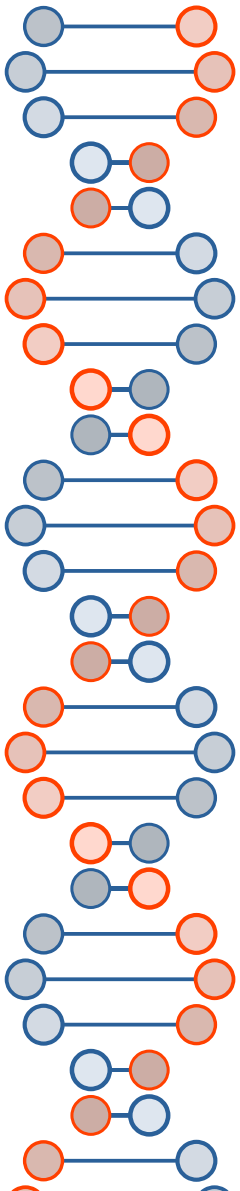
get_time(finish)

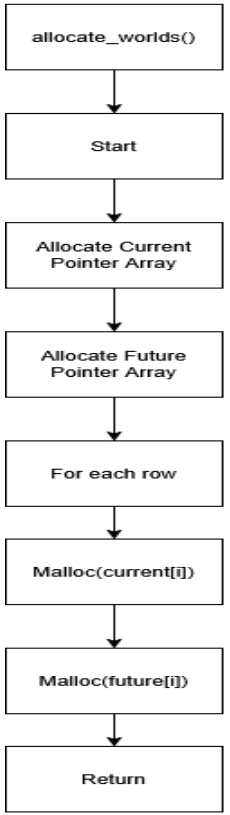
elapsed= finish – start

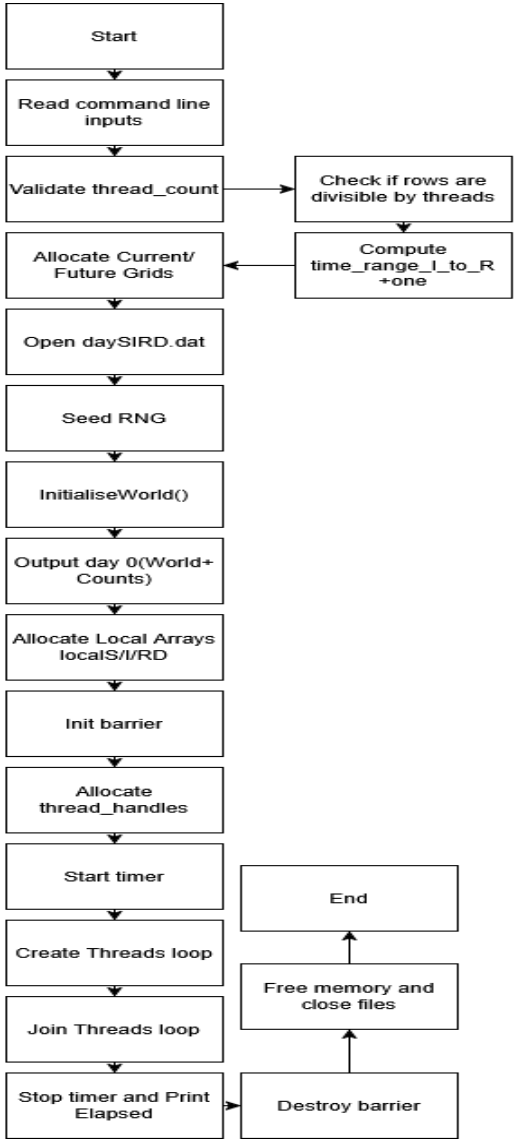
print elapsed

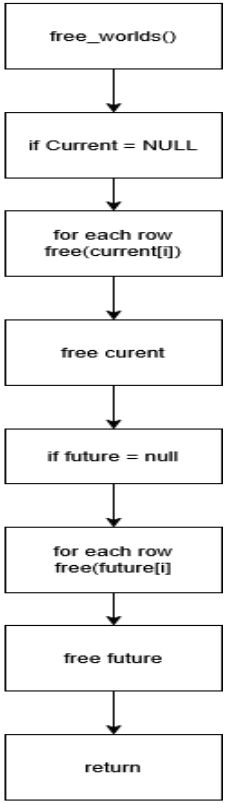
16.Cleanup : destroy barrier, free thread_handles, close fp_daySIRD, free local arrays and free worlds current/future.

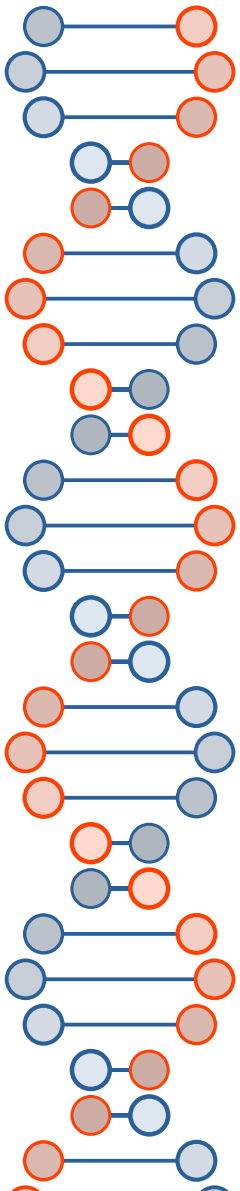
17. Return success




allocate_worlds()
Start
Allocate Current Pointer Array
Allocate Future Pointer Array
For each row
Malloc(current[i])
Malloc(future[i])
Return

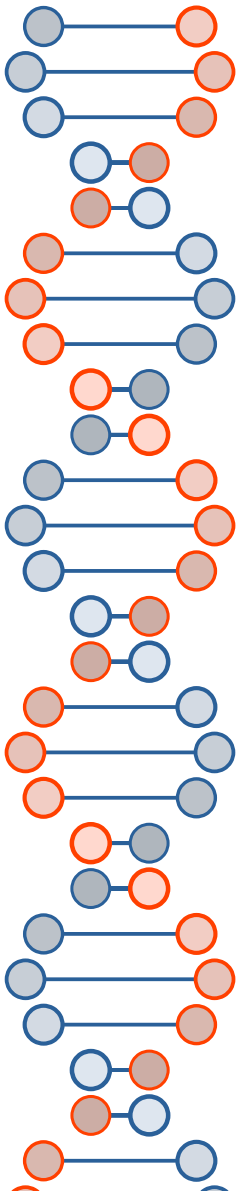

Start
Read command line inputs
Validate thread_count
Check if rows are divisible by threads
Compute time_range_I_to_R +one
Allocate Current/ Future Grids
Open daySIRD.dat
Seed RNG
InitialiseWorld()
Output day 0(World+ Counts)
Allocate Local Arrays localS/I/RD
Init barrier
Allocate thread_handles
Start timer
Create Threads loop
Join Threads loop
Stop timer and Print Elapsed
Destroy barrier
Free memory and close files
End


free_worlds()
if Current = NULL
for each row free(current[i])
free curent
if future = null
for each row free(future[i]
free future
return



Pseudo Code for child_thread.c in parallel_basic

- The following section will be the pseudo code for the child thread used in parallel_basic model, followed by its flow chart.



Includes : stdio, stdlib, time, pthread, timer.h, vars_defs_functions.h

Function child_thread(rank):

1. Convert the argument into an integer thread id:

my_rank = (long) rank

2. Compute row partitions by threads

rows_per_thread = ROWS/ thread_count

start_row = my_rank * rows_per_thread

end_row = start_row + rows_per_thread

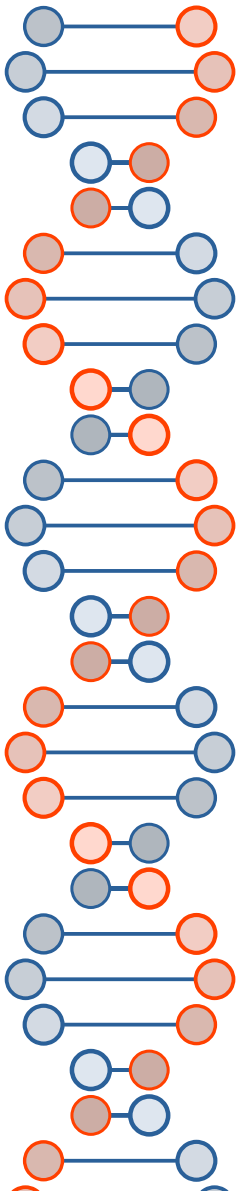
3.Create a local thread random number generator state:

randBuffer is a struct drand48_data

seed = time(null) + my_rank * 10007

srand48_r(seed, randBuffer)

(each thread gets a different stream)



4. Loop over simulation days:

For day = 1 to TOTAL_DAYS:

4.1. initialise local counters for this thread on current:

s = I = r = d = 0

4.2 Update assigned stripe of grid:

For row = start_row to end_row-1:

For col = 0 to COLS-1:

Copy current cell into future cell (default aka no change”):

future[row][col] = current[row][col]

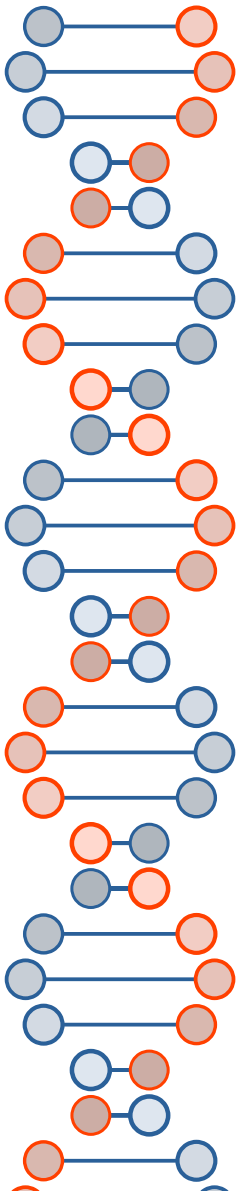
Apply rules based on current state:

if state == 'S':

n = countInfectedNeighbours(row, col, current)

decide_S_to_I(row, col, n, future, &randBuffer)

else if state == 'I':



decide_I_to_RorD(row, col, current, future, &randBuffer)

else if state == 'D':

decide_D_to_Empty(row, col, current, future)

Else: do nothing

count state of updated cell (future)

increment s,i,r,d based on future[row][col].state

4.3 store the daily counts into shared arrays:

localS [my_rank] = s

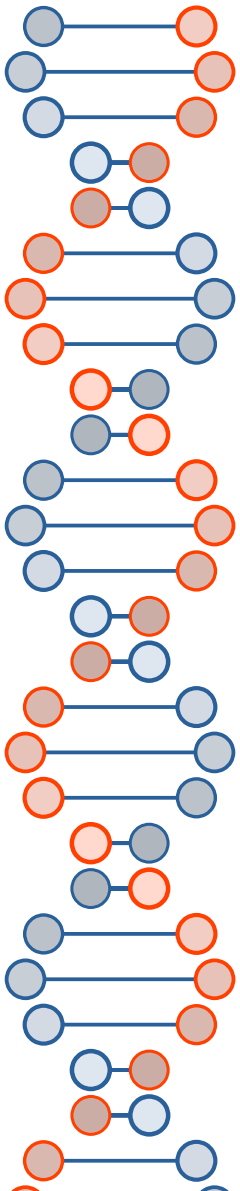
localI [my_rank] = I

localR [my_rank] = r

localD [my_rank] = d

Barrier #1: wait for all threads to finish computing the future grid:

pthread_barrier_wait(&barrier)



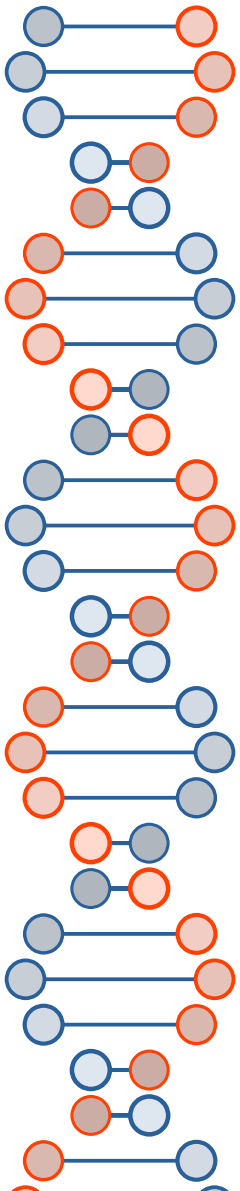
Serial section executed by ONE thread (rank 0):

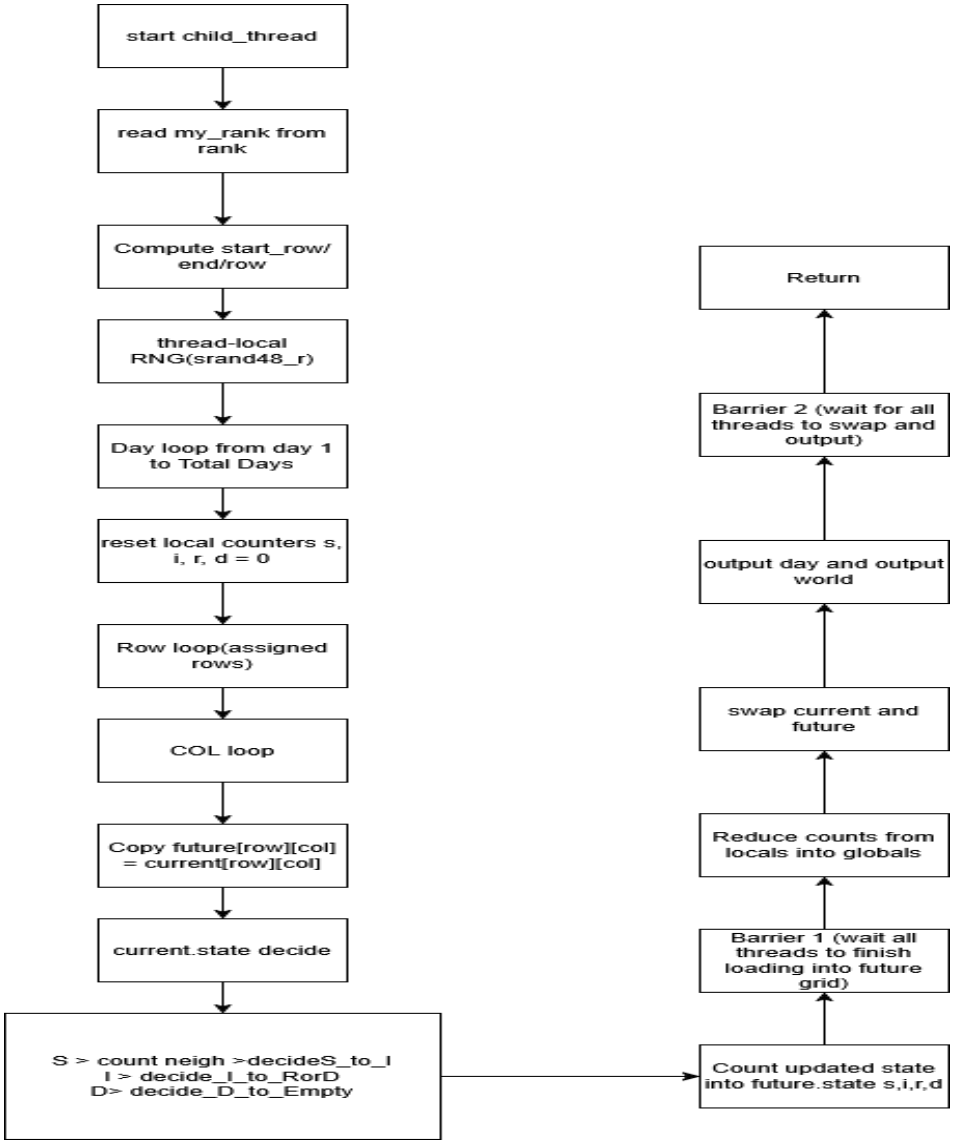
- Reduce all local counts into global totals numS/numI/numR/numD
- Swap current and future pointers (so the next day reads the updated grid)
- Write outputDaySIRD for this day
- outputWorld snapshot every OUTPUT_FREQ days
- print debug “Finished day”

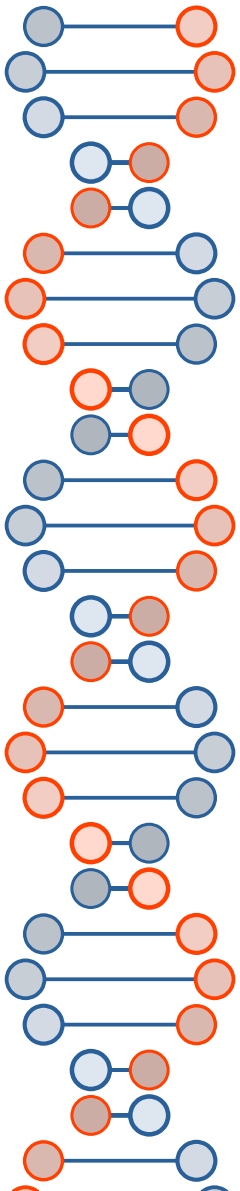
Barrier #2 = all threads wait until swap and output are complete:

pthread_barrier_wait(barrier)

Return null

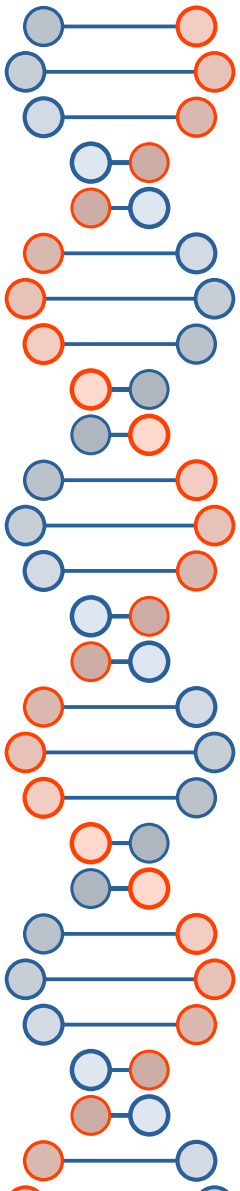



start child_thread
read my_rank from rank
Compute start_row/ end/row
thread-local RNG(srand48_r)
Day loop from day 1 to Total Days
reset local counters s, i, r, d = 0
Row loop(assigned rows)
COL loop
Copy future[row][col] = current[row][col]
current.state decide
S > count neigh >decideS_to_I
I > decide_I_to_RorD
D> decide_D_to_Empty
Count updated state into future.state s,i,r,d
Barrier 1 (wait all threads to finish loading into future grid)
Reduce counts from locals into globals
swap current and future
output day and output world
Barrier 2 (wait for all threads to swap and output)
Return



Pseudo Code for Parallel_latent

- Next slides will be allocated to the pseudo code for parallel_latent.c
- It’s very similar to parallel_basic.c, just having the addition of the E state, SERID file instead of SIRD for ouput, and the S to E and E to I transitions.



includes standard libraries : stdio, stdlib, time, pthread

timer.h for get_time macro

vars_defs_functions.h for macros, variables and functions control

Global/File variables:

time_range_I_to_R_plusOne

time_range_E_to_I_plusOne

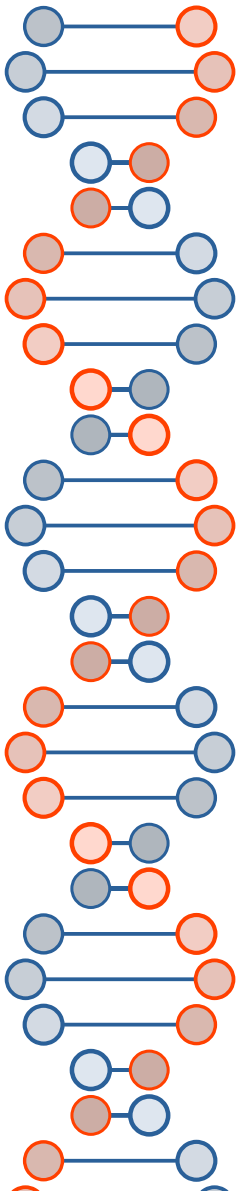
thread_count

barrier// pthread barrier used to synchronise threads each day

current, future

fp_daySEIRD// file pointer to write sird data

localS, localE localI, localR, localD// arrays per thread_count to avoid race conditions



Function prototype:

child_thread(rank) //thread function defined in other file

Function allocate_worlds():

Allocate array of rows pointers for current and future

For each row I :

allocate COLS CELL for current[i]

allocate COLS CELL for future[i]

if malloc fails > print error and exit

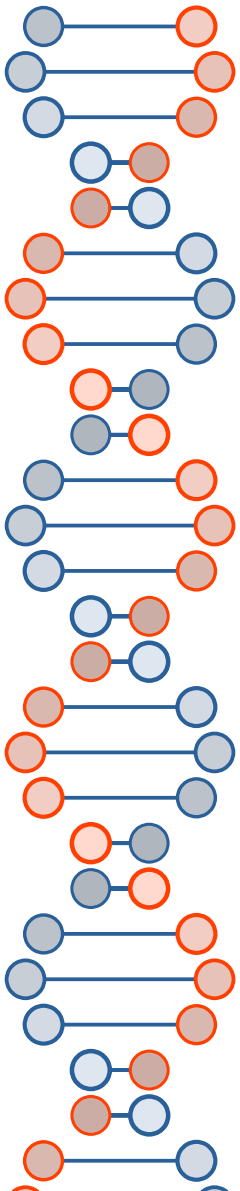
Function free_worlds():

if current exists:

for each row I: free(current[i])

free current and set to NULL

do the same for future



Function main(argc, argv):

1. validate input arguments:

if argc < 2: print usage and exit

thread_count = atoi(argv[1])

if thread_count <= 0: print error and exit

if rows are not divisible by thread_count: print error and exit

2. initialisation constant:

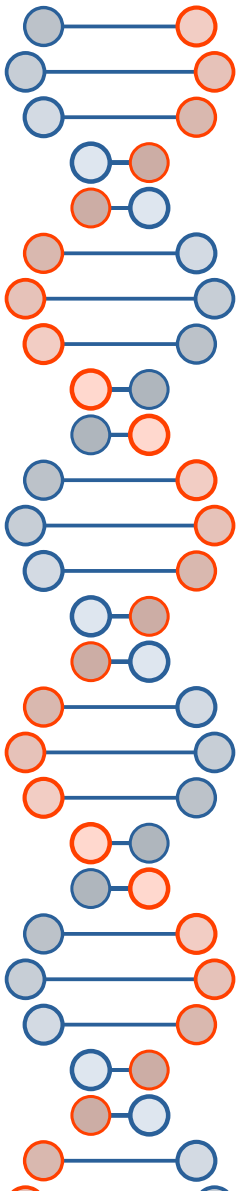
time range I to R plus one = Max_Time_I_to_R – Min_Time_I_to_R + 1

time range E to I plus one = Max_Time_E_to_I – Min_Time_E_to_I + 1

3. Allocate the two worlds:

allocate_worlds()

4. Set day 0 global counters for numS numE, numI, numR, numD = 0



5. Open output file data/daySEIRD.dat

6. Seed random number generator

srand48(time(null))

7. Initialise world

intialise_world(current, future, numS, numI)

8.Output day 0:

OutputWorld(0, current)

outputdaySEIRD(fp_daySEIRD, 0, numS, numE, numI, numR, numD)

9. Allocate thread count arrays:

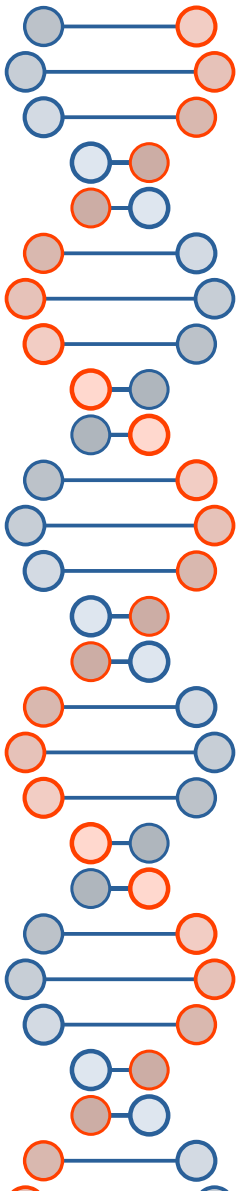
localS,local E, localI, localR, localD = calloc(thread_count)

10. Initialise barrier for thread_count threads:

pthread_barrier_init()

11. Allocate thread handle array:

thread_handles = malloc(thread_count)



12. Start timer:

GET_TIME(START)

13. Create threads

for thread id in 0 >> thread_count -1:

pthread_create(handles[thread], child_thread)

14. Join threads

for thread id in 0 >> thread_count -1:

pthread_join(thread_handles[thread]

15. Stop timer and print:

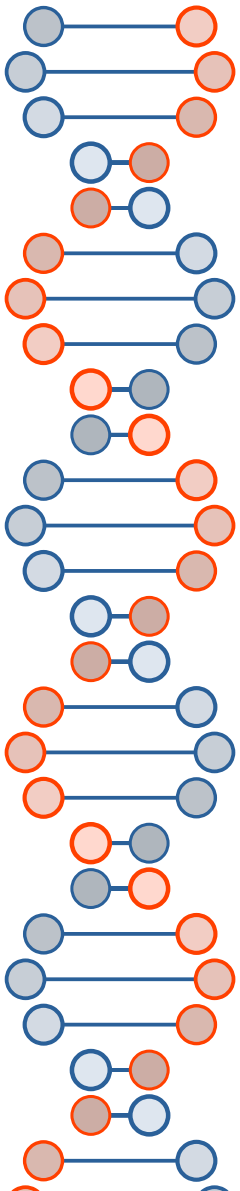
get_time(finish)

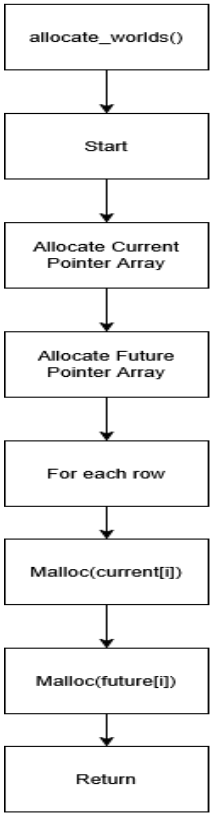
elapsed= finish – start

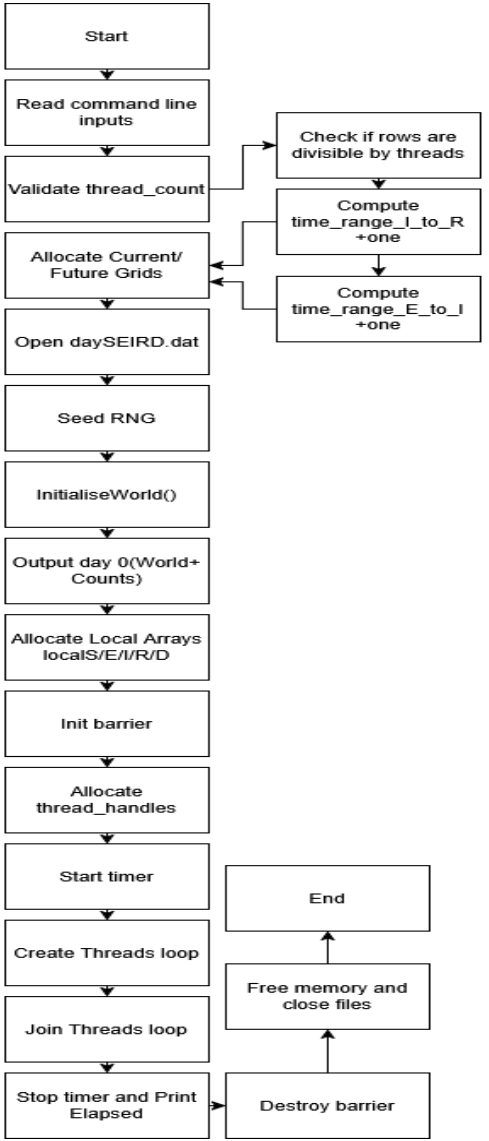
print elapsed

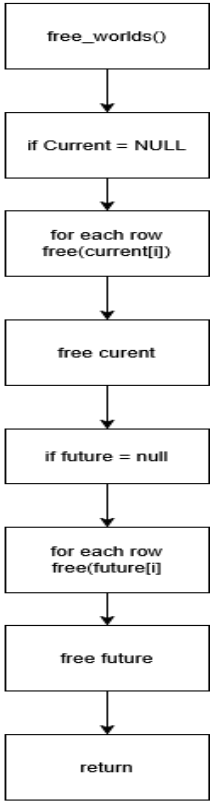
16.Cleanup : destroy barrier, free thread_handles, close fp_daySEIRD, free local arrays and free worlds current/future.

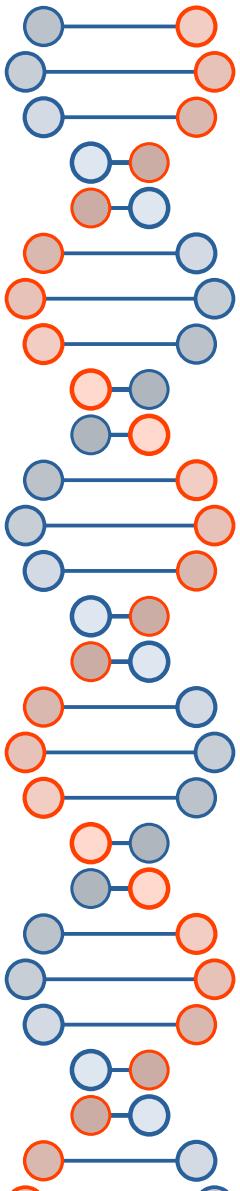
17. Return success




allocate_worlds()
Start
Allocate Current Pointer Array
Allocate Future Pointer Array
For each row
Malloc(current[i])
Malloc(future[i])
Return

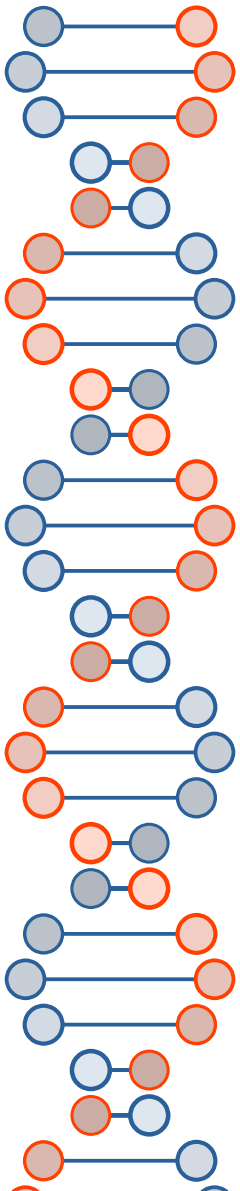

Start
Read command line inputs
Validate thread_count
Check if rows are divisible by threads
Compute time_range_I_to_R +one
Allocate Current/ Future Grids
Compute time_range_E_to_I +one
Open daySEIRD.dat
Seed RNG
InitialiseWorld()
Output day 0(World+ Counts)
Allocate Local Arrays localS/E/I/R/D
Init barrier
Allocate thread_handles
Start timer
Create Threads loop
Join Threads loop
Stop timer and Print Elapsed
Destroy barrier
Free memory and close files
End


free_worlds()
if Current = NULL
for each row free(current[i])
free curent
if future = null
for each row free(future[i]
free future
return



Pseudo Code for child_thread in parallel_latent

- The following slides will be the pseudo code used for the development of child_thread.c in the parallel_latent model.
- As with the main function, the logic between child thread in basic model and latent model is the same, the only difference being the addition of the E state, using the SEIRD.dat output and using S to E and E to I transitions, with a counter for E to I.



Includes : stdio, stdlib, time, pthread, timer.h, vars_defs_functions.h

Function child_thread(rank):

1. Convert the argument into an integer thread id:

my_rank = (long) rank

2. Compute row partitions by threads

rows_per_thread = ROWS/ thread_count

start_row = my_rank * rows_per_thread

end_row = start_row + rows_per_thread

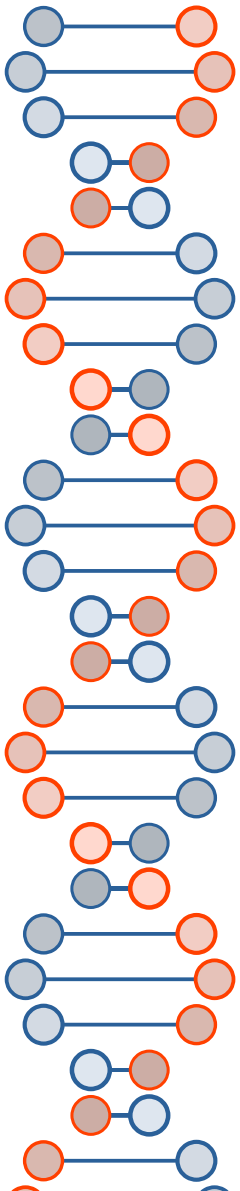
3.Create a local thread random number generator state:

randBuffer is a struct drand48_data

seed = time(null) + my_rank * 10007

srand48_r(seed, randBuffer)

(each thread gets a different stream)



4. Loop over simulation days:

For day = 1 to TOTAL_DAYS:

4.1. initialise local counters for this thread on current:

s = E = I = r = d = 0

4.2 Update assigned stripe of grid:

For row = start_row to end_row-1:

For col = 0 to COLS-1:

Copy current cell into future cell (default aka no change”):

future[row][col] = current[row][col]

Apply rules based on current state:

if state == 'S':

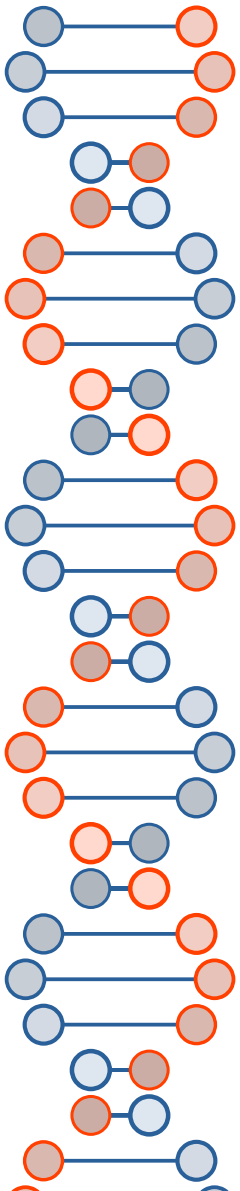
n = countInfectedNeighbours(row, col, current)

decide_S_to_E(row, col, n, future, &randBuffer)

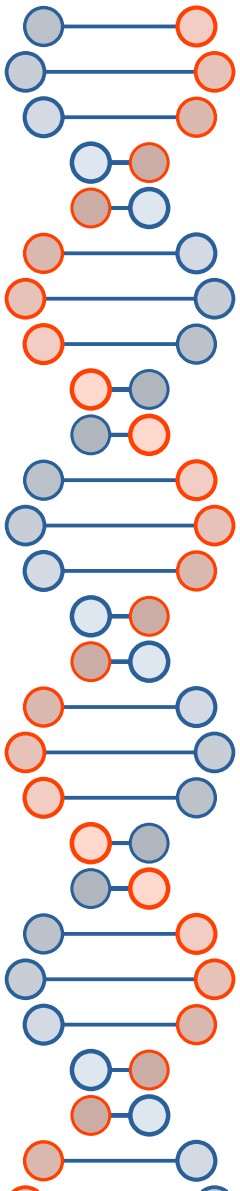
else if state == 'E':

decide_E_to_I(row, col, current, future)

else if state == ‘I’



- decide_I_to_RorD(row, col, current, future, &randBuffer)
- else if state == 'D':
- decide_D_to_Empty(row, col, current, future)
- Else: do nothing
- count state of updated cell (future)
- increment s,e,i,r,d based on future[row][col].state
-
- 4.3 store the daily counts into shared arrays:
- localS [my_rank] = s
- localE [my_rank] = e
- localI [my_rank] = I
- localR [my_rank] = r
- localD [my_rank] = d
- Barrier #1: wait for all threads to finish computing the future grid:
- pthread_barrier_wait(&barrier)



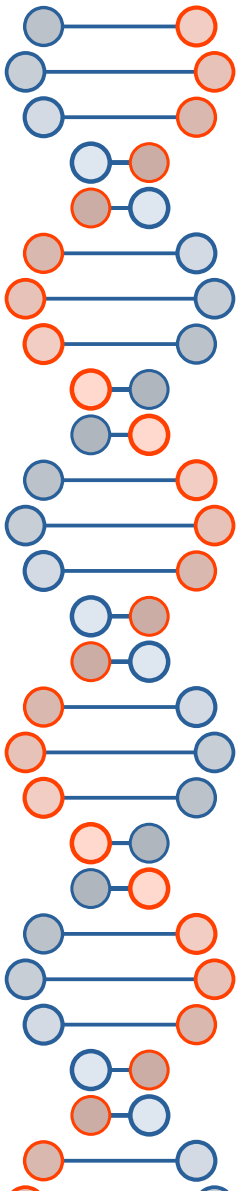
Serial section executed by ONE thread (rank 0):

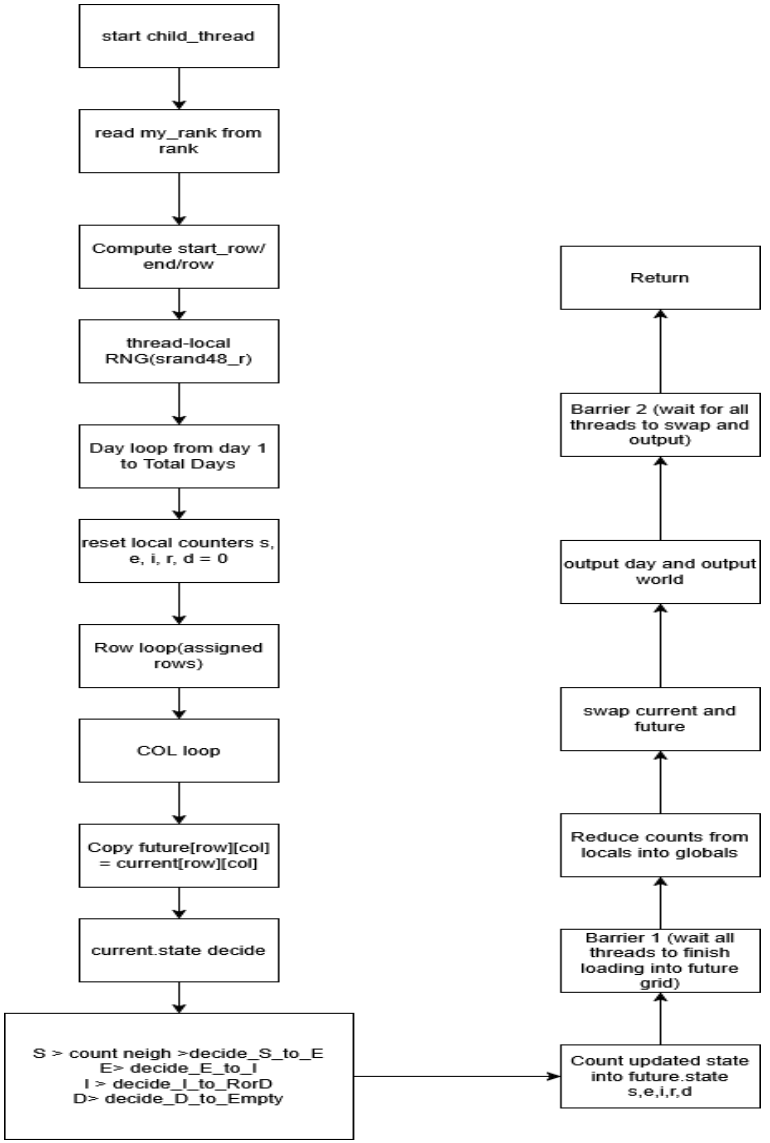
- Reduce all local counts into global totals numS/numE/numI/numR/numD
- Swap current and future pointers (so the next day reads the updated grid)
- Write outputDaySEIRD for this day
- outputWorld snapshot every OUTPUT_FREQ days
- print debug “Finished day”

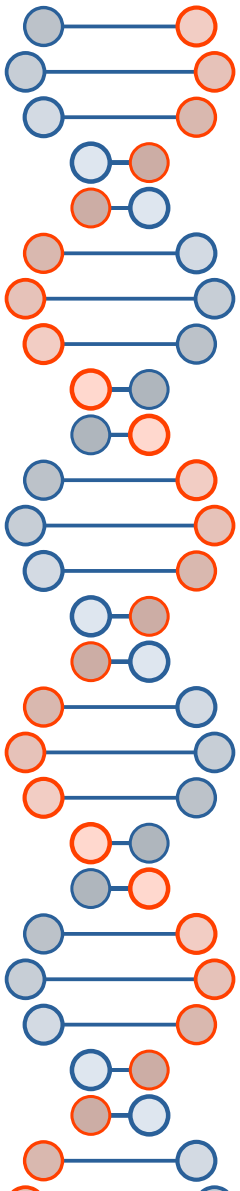
Barrier #2 = all threads wait until swap and output are complete:

pthread_barrier_wait(barrier)

Return null

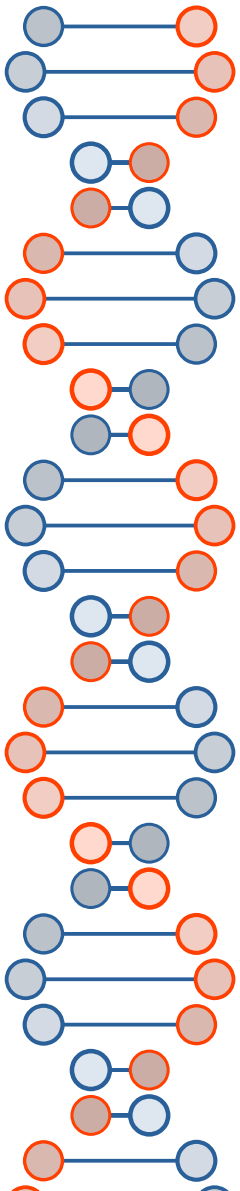



start child_thread
read my_rank from rank
Compute start_row/ end/row
thread-local RNG(srand48_r)
Day loop from day 1 to Total Days
reset local counters s, e, i, r, d = 0
Row loop(assigned rows)
COL loop
Copy future[row][col] = current[row][col]
current.state decide
S > count neigh >decide_S_to_E
E> decide_E_to_I
I > decide_I_to_RorD
D> decide_D_to_Empty
Count updated state into future.state s,e,i,r,d
Barrier 1 (wait all threads to finish loading into future grid)
Reduce counts from locals into globals
swap current and future
output day and output world
Barrier 2 (wait for all threads to swap and output)
Return



Testing of Parallel Programs

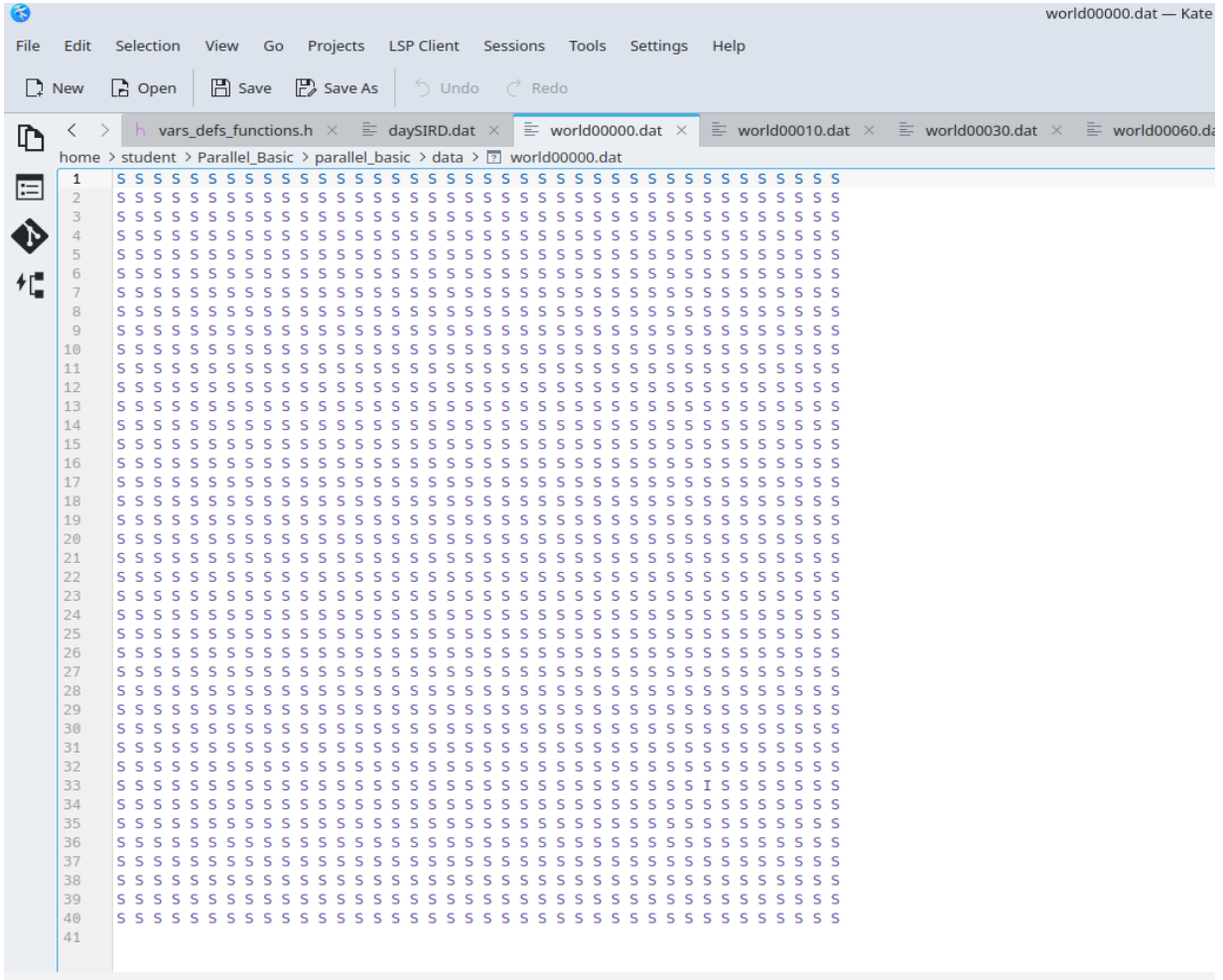
- For testing, to verify correctness I compared daySEIRD.dat with the counts from worldXXXXX.dat.
- Tested serial vs parallel with thread counts : 1,2,4,8.
- Used identical paramters and random seeds.
- Timing was measured using GET_TIME() around simulation loop.
- All results of those tests will be shown in the next chapter which is about the results of the simulations.



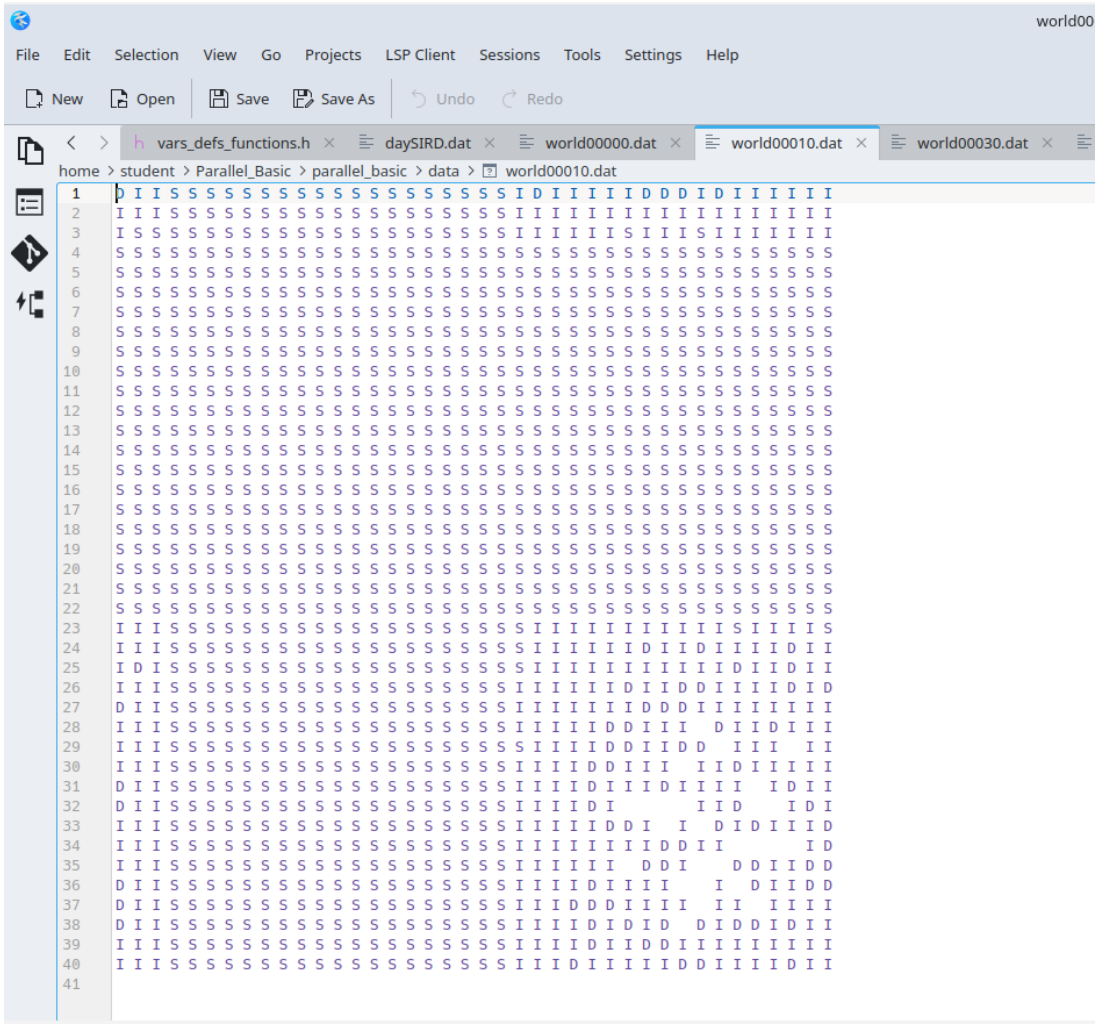
Initial Conditions for visualisation

- Grid size is 40 x 40.
- 60 total days.
- Output frequency is every 10 days.

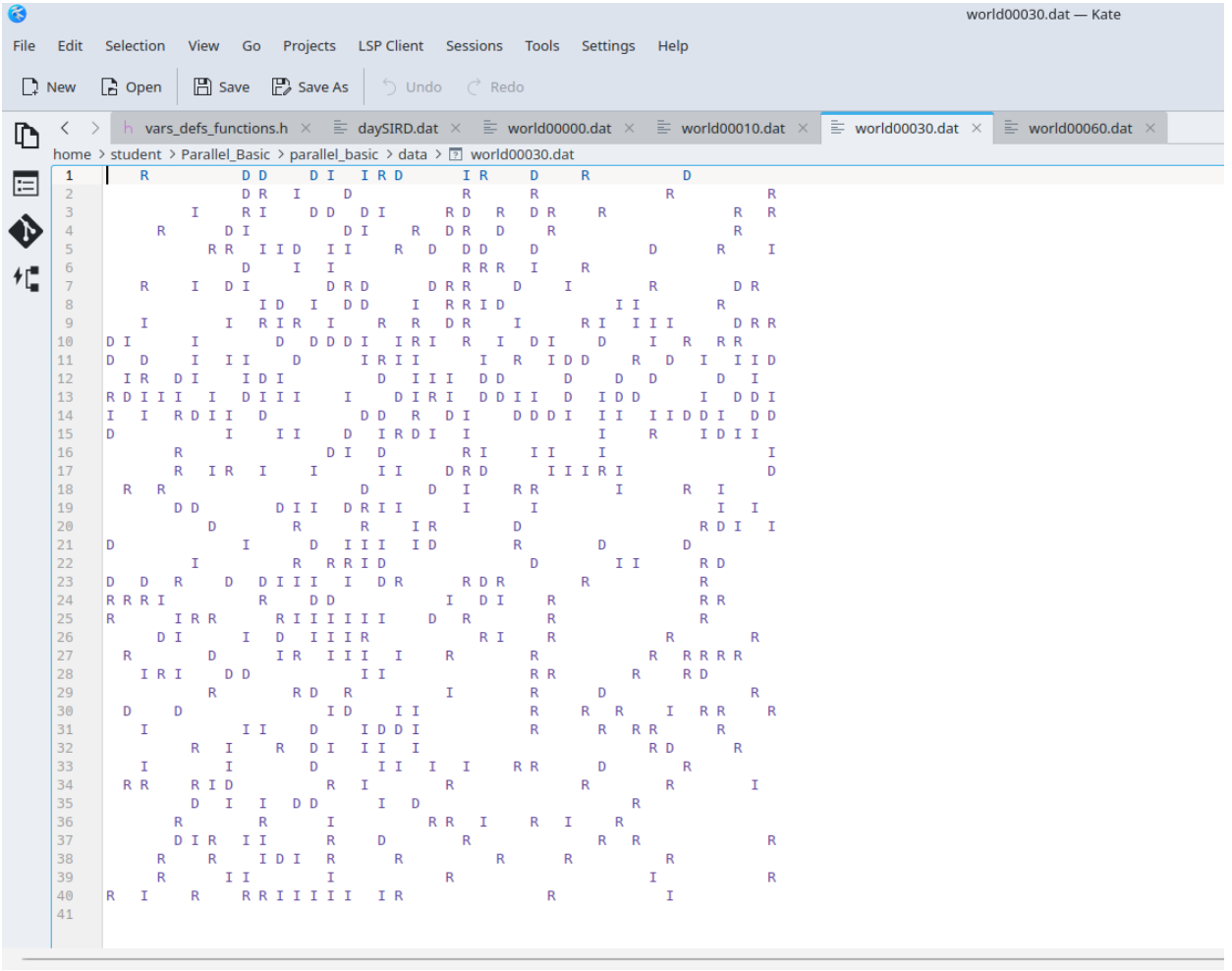
Day 0 for Parallel_Basic



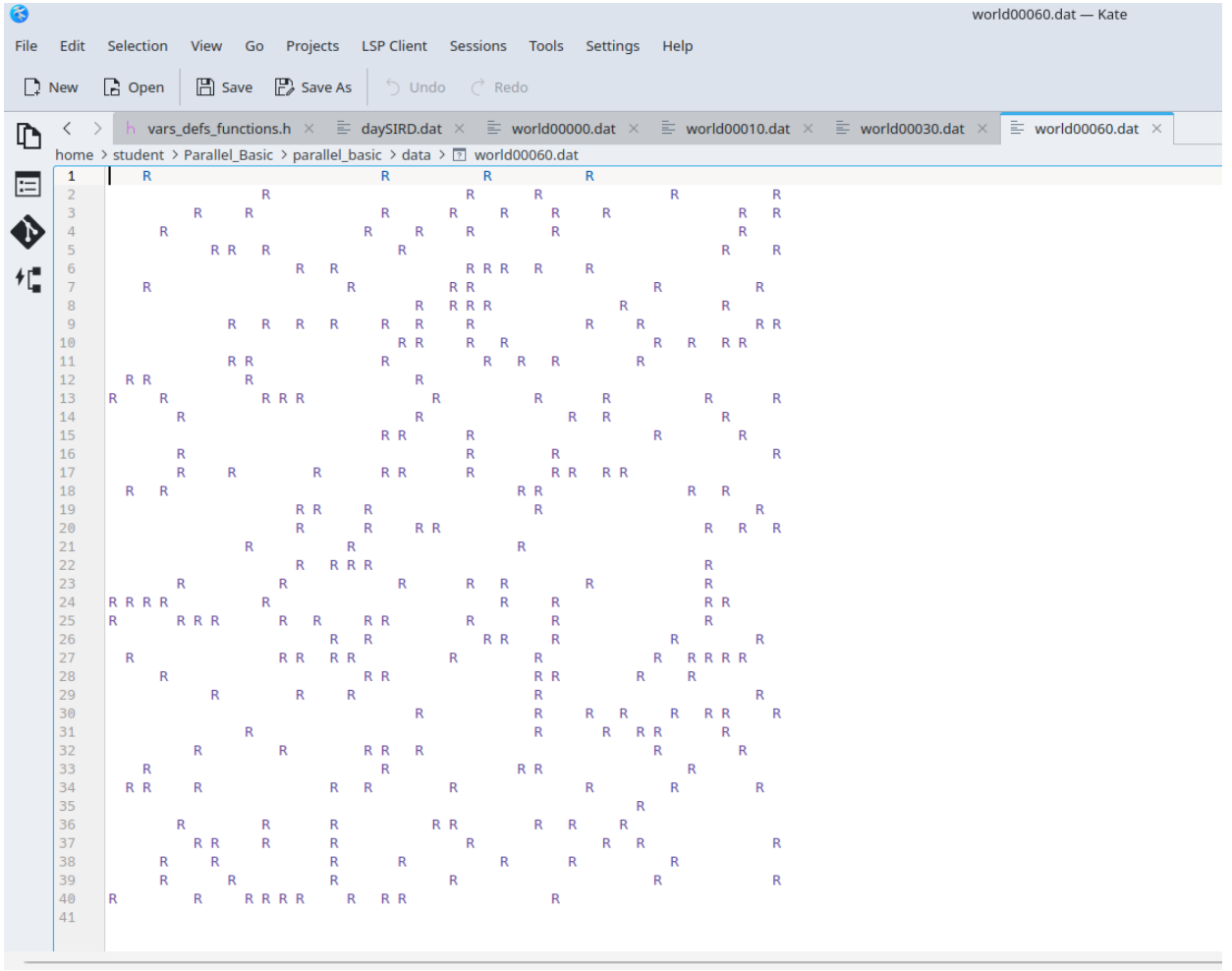
Day 10 for Parallel Basic



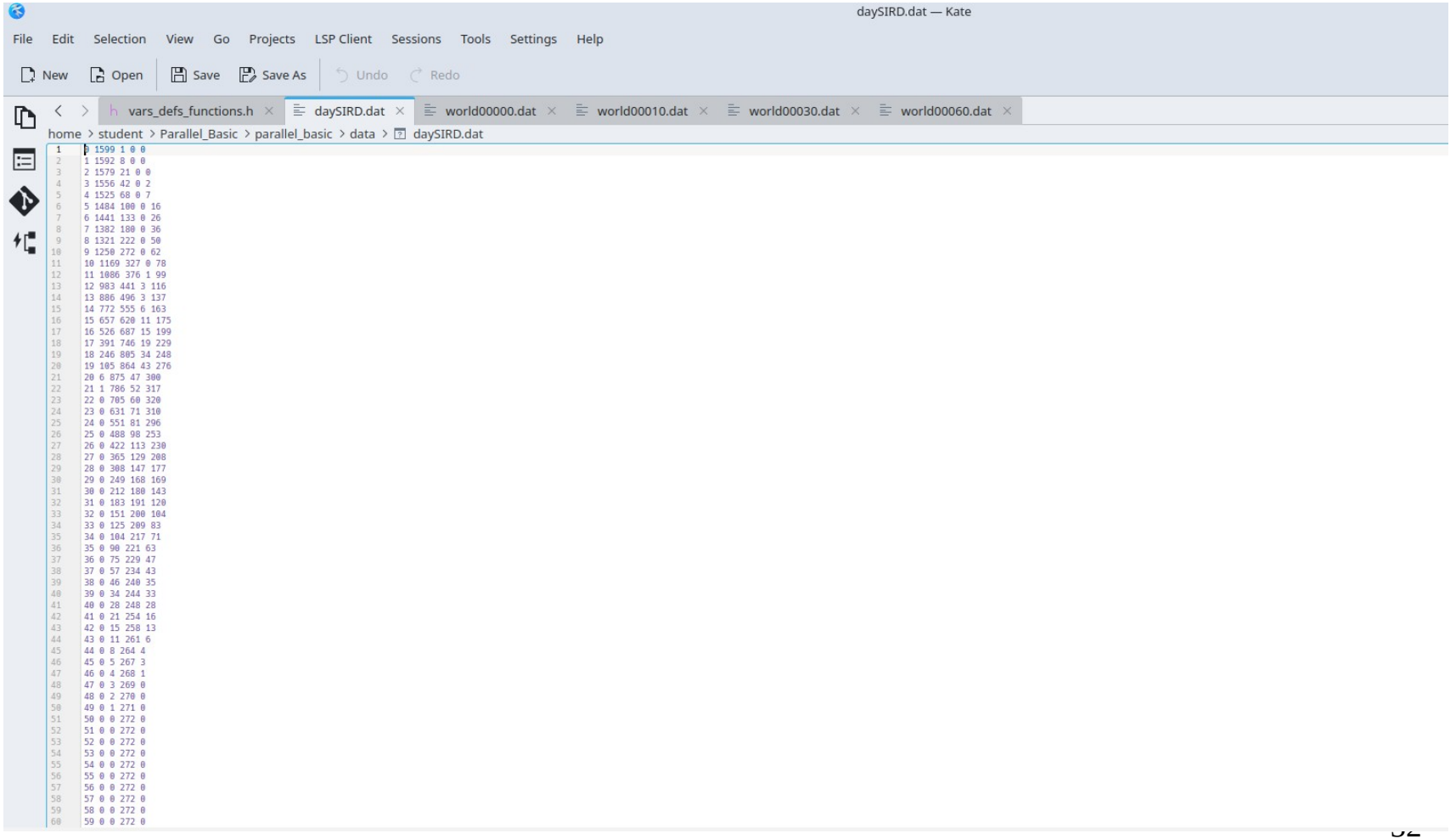
Day 30 for Parallel Basic



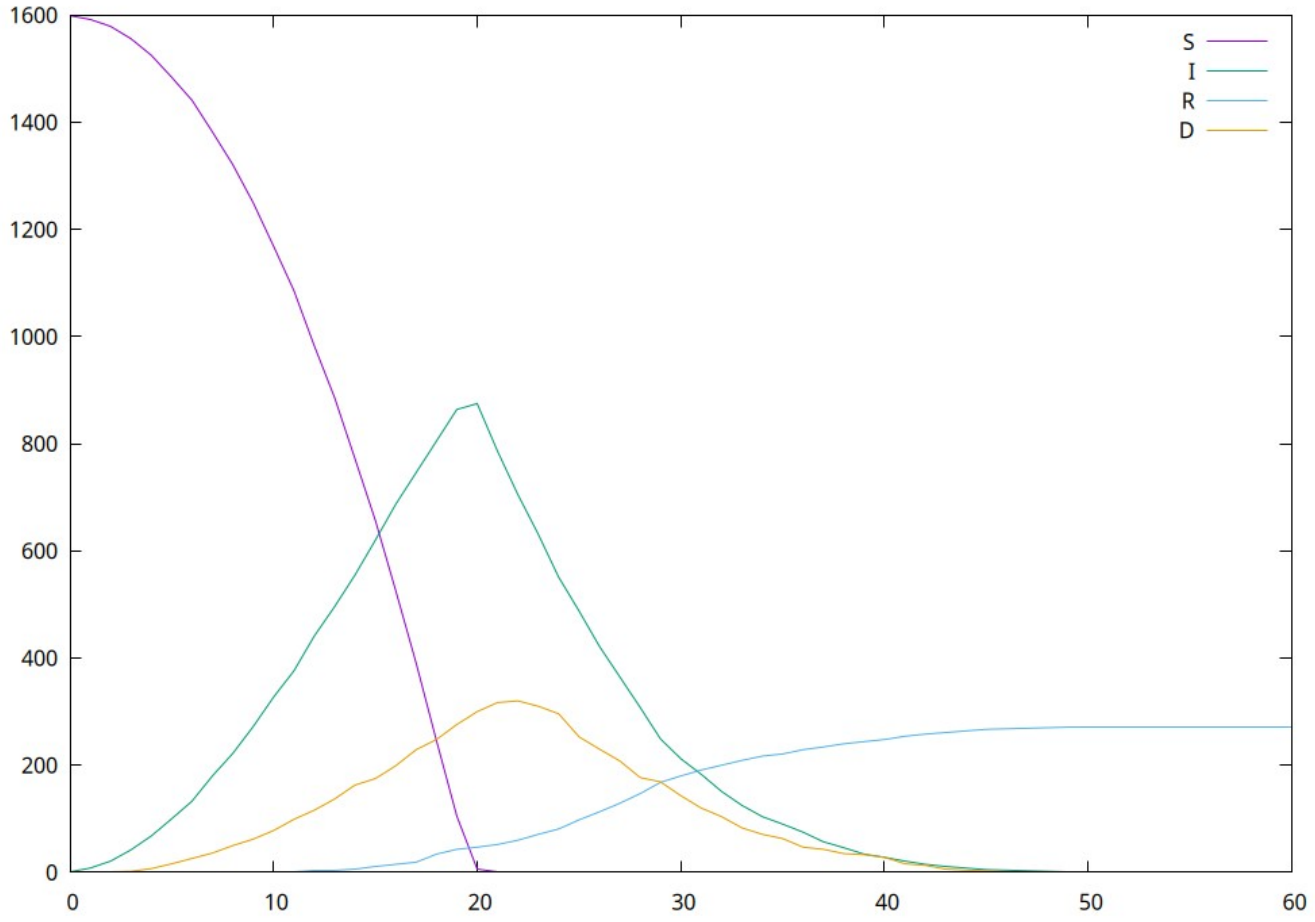
Day 60 for Parallel Basic



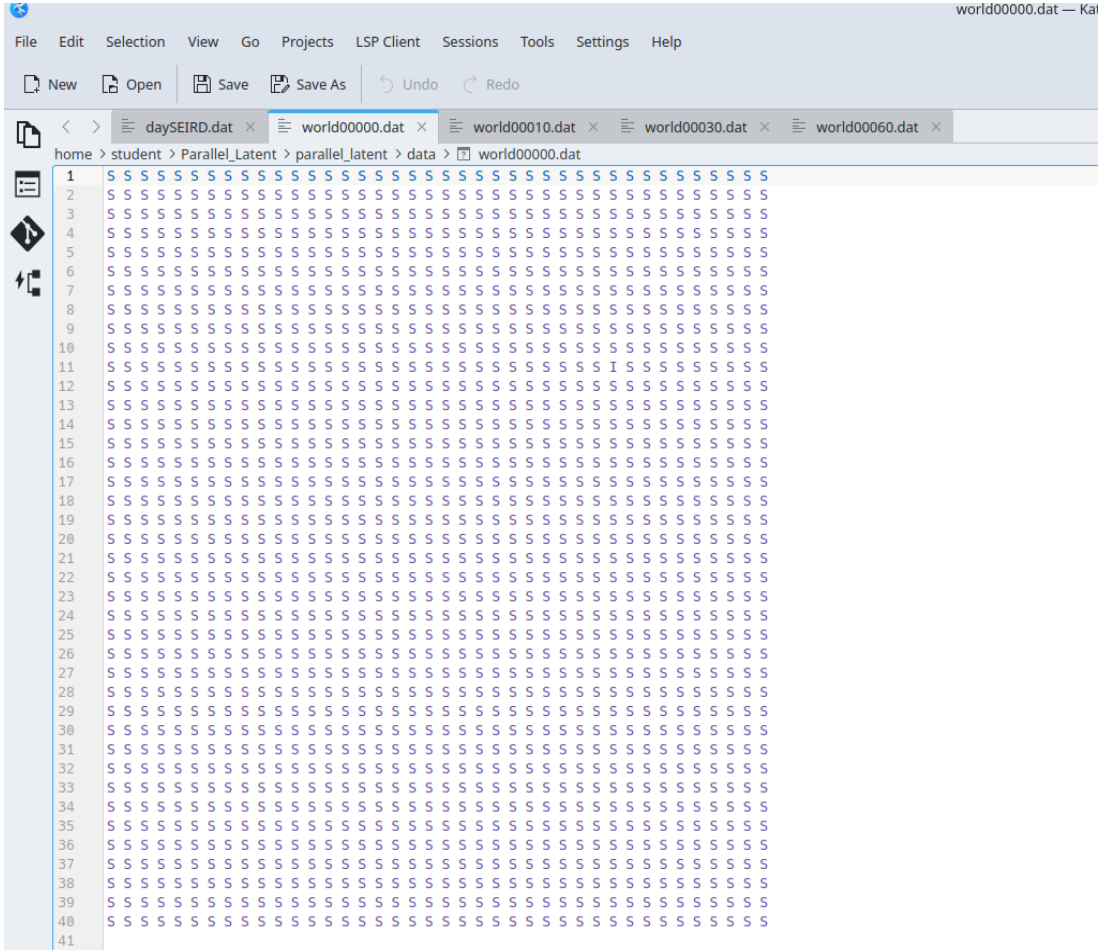
DaySIRD.dat for Parallel Basic



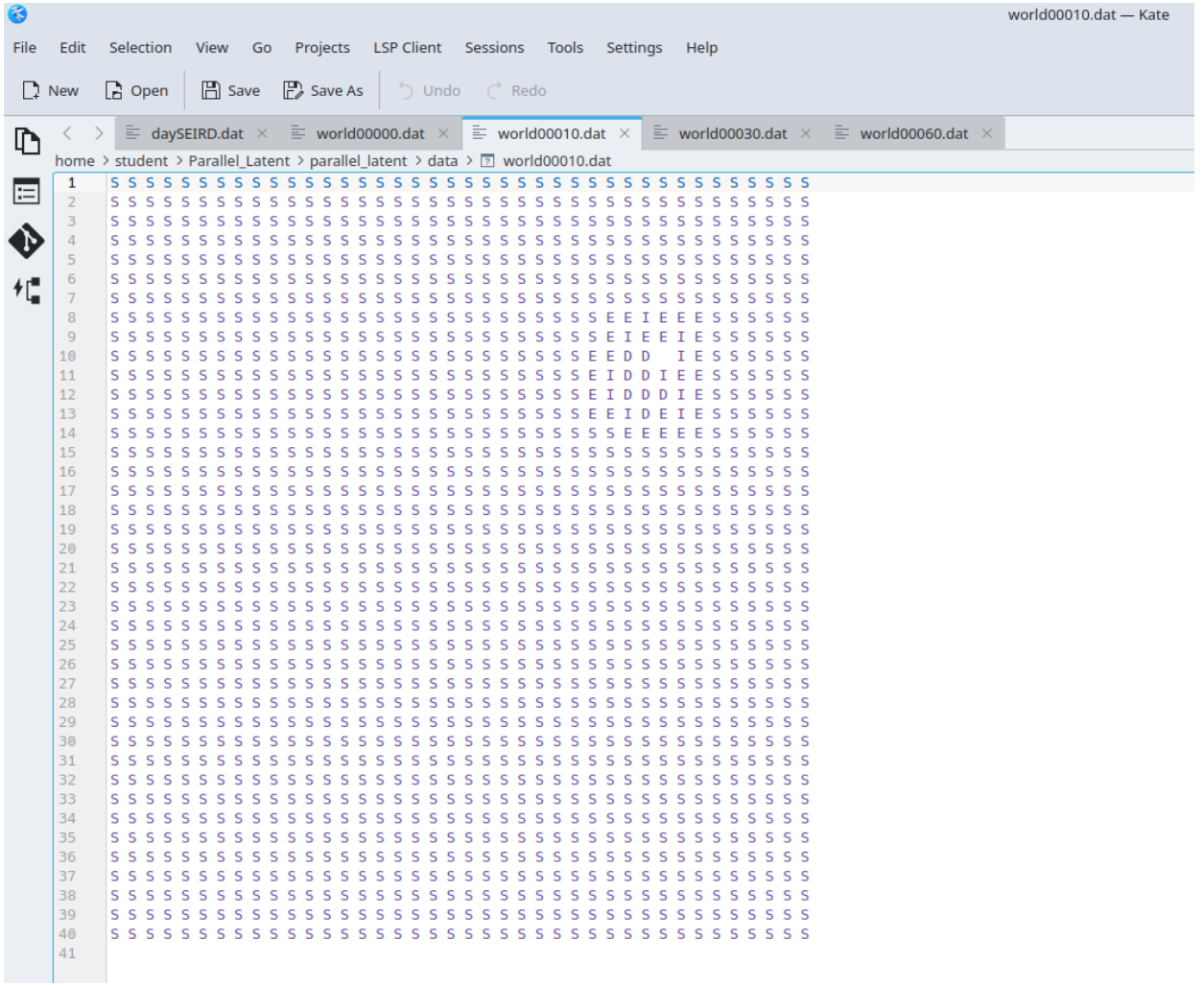
GNU plot graph for Parallel Basic



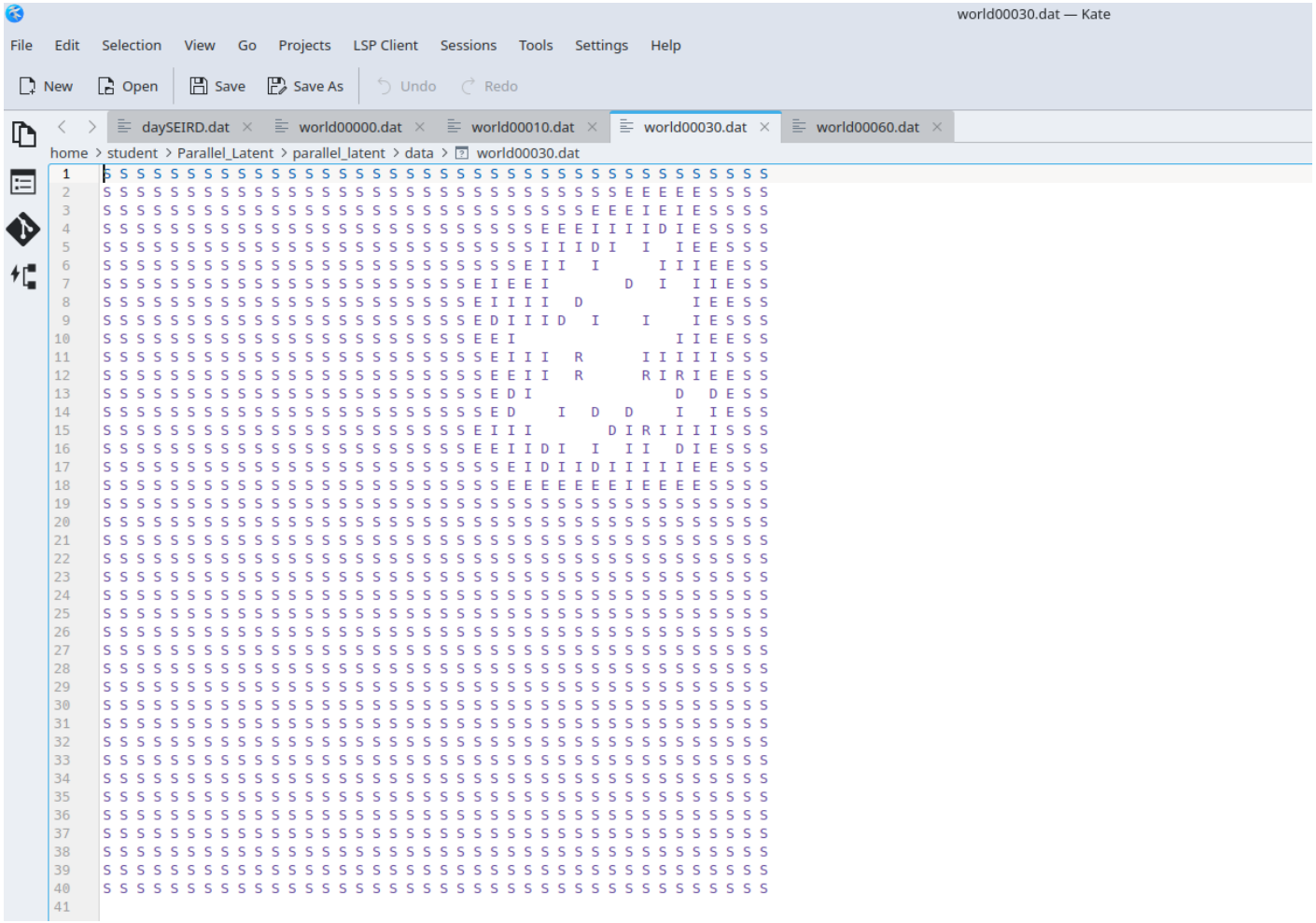
Day 0 for Parallel Latent



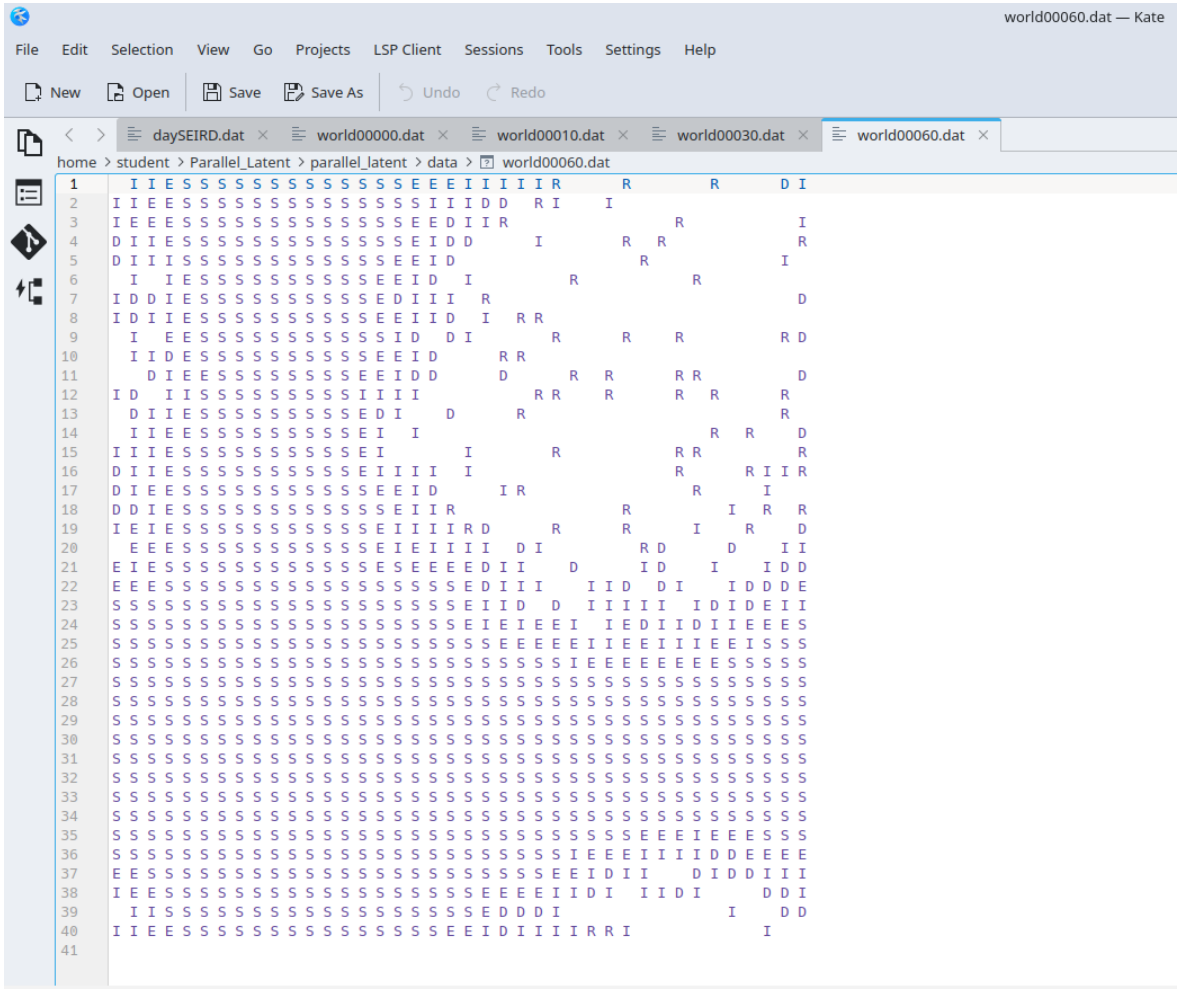
Day 10 for Parallel Latent



Day 30 for Parallel Latent



Day 60 for Parallel Latent



DaySEIRD.dat for Parallel Latent

daySEIRD.dat — Kate

File Edit Selection View Go Projects LSP Client Sessions Tools Settings Help

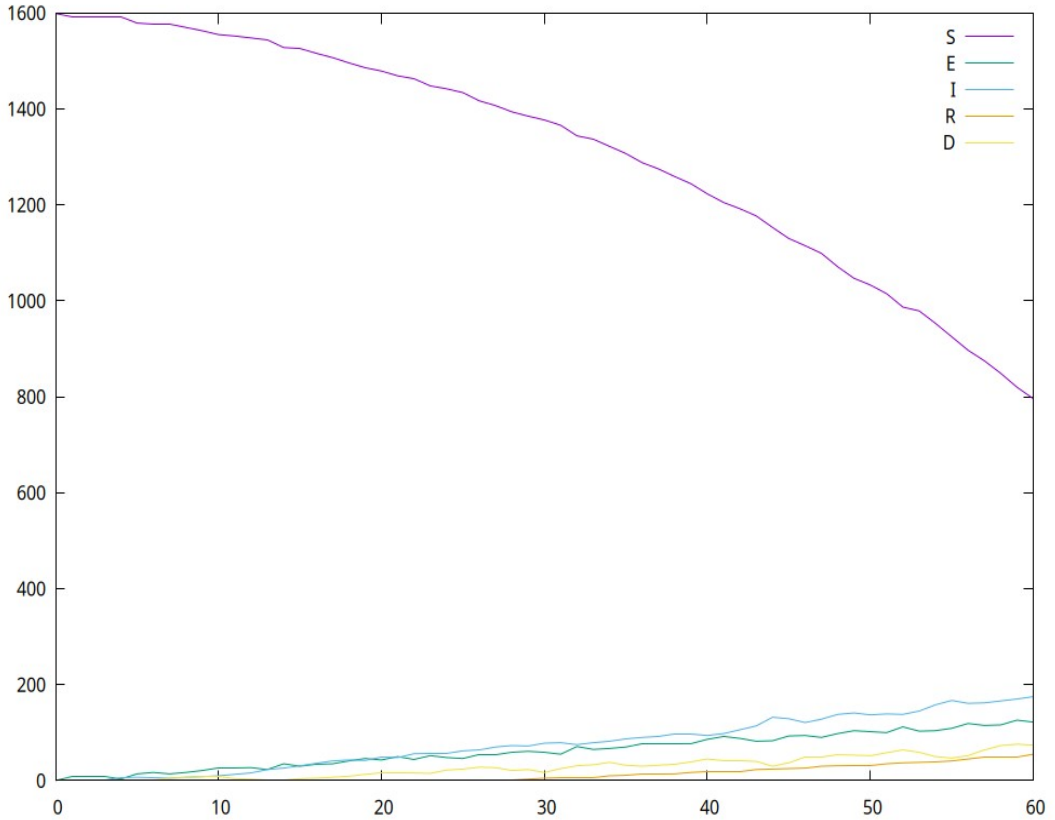
New Open Save Save As Undo Redo

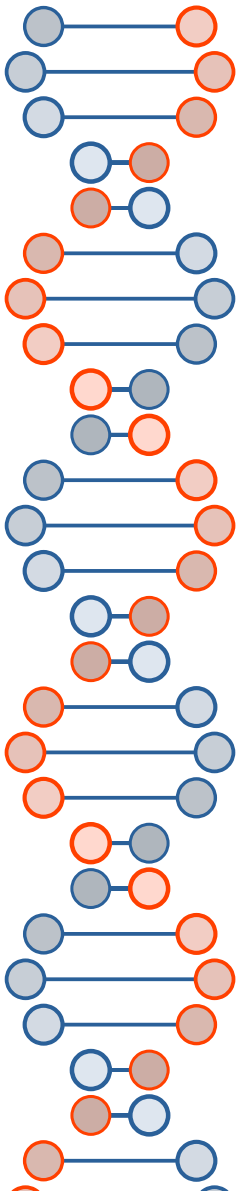
daySEIRD.dat | world00000.dat | world00010.dat | world00030.dat | world00060.dat

home > student > Parallel_Latent > parallel_latent > data > daySEIRD.dat

```
0 1599 0 1 0 0
1 1591 8 1 0 0
2 1591 8 1 0 0
3 1591 8 1 0 0
4 1591 3 6 0 0
5 1579 14 7 0 0
6 1576 17 6 0 1
7 1576 14 5 0 5
8 1570 17 7 0 6
9 1563 21 8 0 8
10 1555 26 10 0 8
11 1552 26 13 0 4
12 1548 27 16 0 3
13 1544 23 23 0 2
14 1528 35 26 0 2
15 1526 30 31 0 4
16 1516 34 36 0 5
17 1507 35 41 0 7
18 1496 41 43 0 9
19 1486 46 42 0 13
20 1479 43 49 0 15
21 1469 50 48 1 15
22 1463 44 56 1 16
23 1448 52 57 2 15
24 1442 48 57 2 22
25 1434 46 62 2 24
26 1417 53 64 2 28
27 1407 53 70 2 27
28 1394 59 73 2 21
29 1385 61 72 3 23
30 1377 59 78 5 17
31 1366 55 79 7 25
32 1344 71 75 7 31
33 1337 65 79 7 33
34 1322 67 82 10 38
35 1307 70 87 11 32
36 1288 76 90 13 30
37 1275 76 92 13 32
38 1259 76 98 14 34
39 1244 77 98 17 39
40 1223 86 94 19 45
41 1205 92 98 19 42
42 1192 88 106 19 42
43 1177 82 114 23 40
44 1153 83 132 24 30
45 1130 93 129 25 37
46 1115 94 121 26 49
47 1099 90 128 30 49
48 1071 98 138 31 54
49 1047 104 141 32 53
50 1033 102 137 32 52
51 1015 100 139 35 58
52 987 112 138 37 64
53 979 103 145 38 59
54 953 104 158 39 50
55 925 109 167 41 47
56 897 119 161 45 52
57 875 115 162 49 64
58 849 116 166 50 73
59 820 126 170 50 76
```

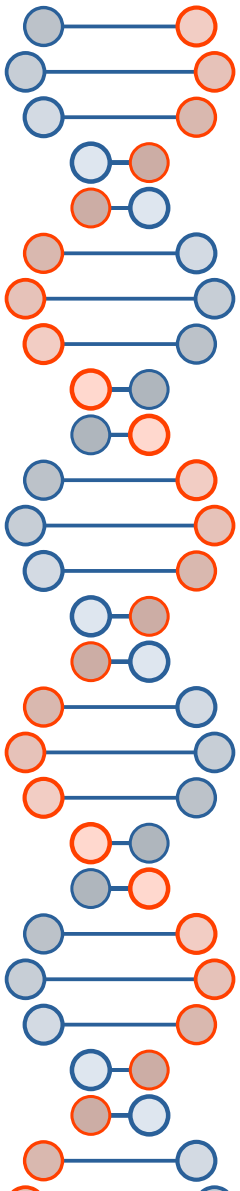
GNU plot graph for Parallel Latent





Conclusion for Visualisation testing

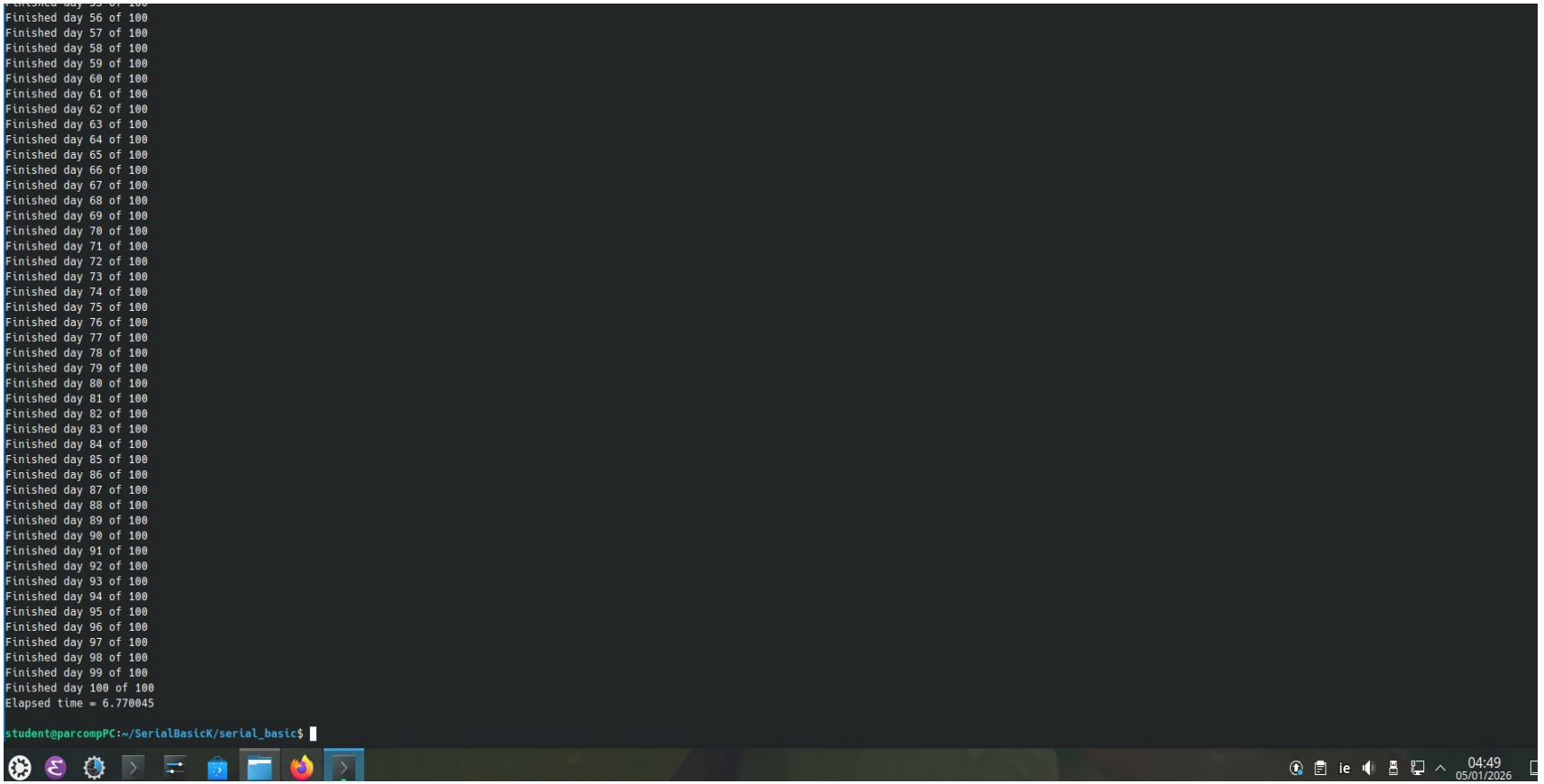
- Both programs run well and the results are to be expected.
- The spread of the virus is slower in the latent version as we have an extra ‘E’ state that delays the spread by having a certain period of time added to it before it becomes Infectious.



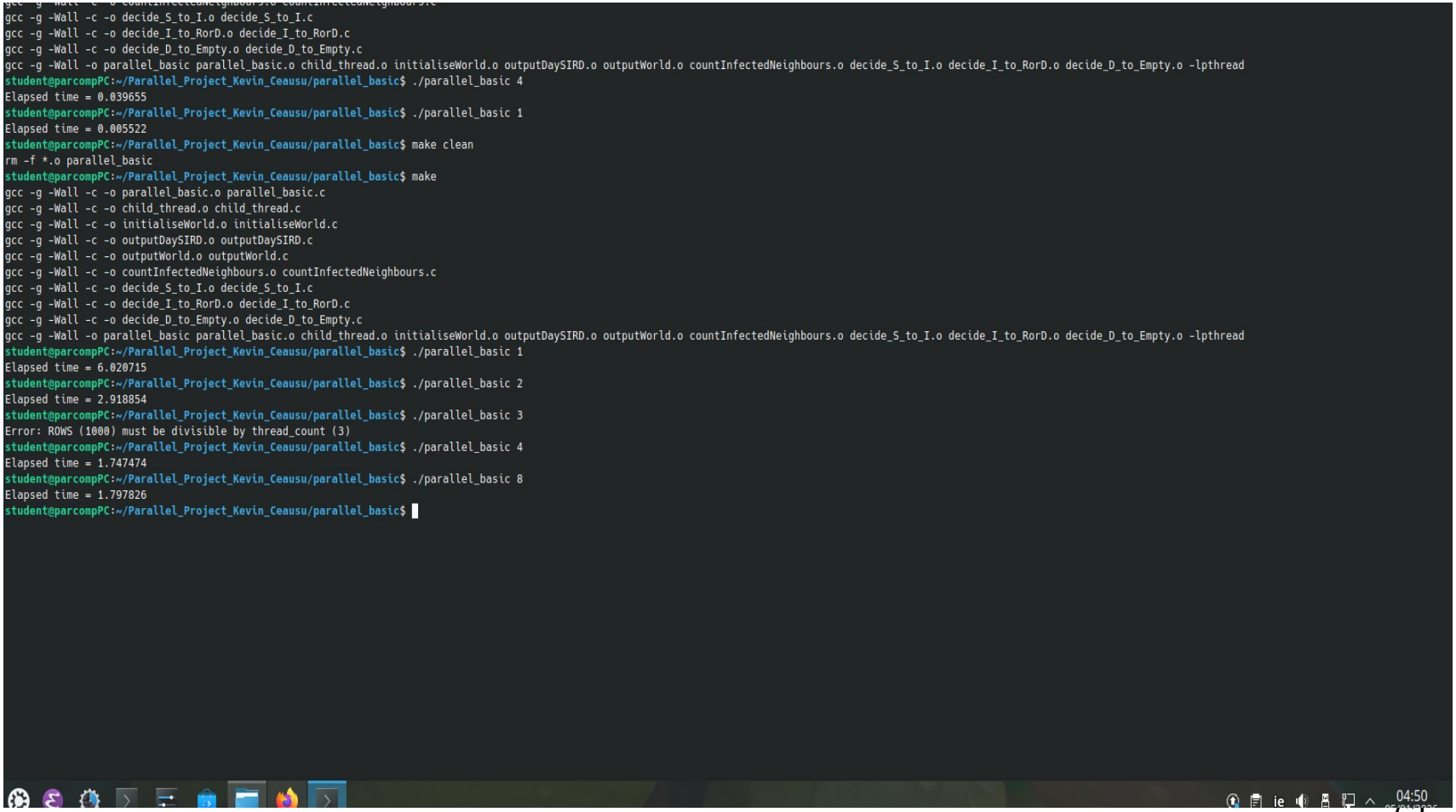
Speed testing – Conditions used

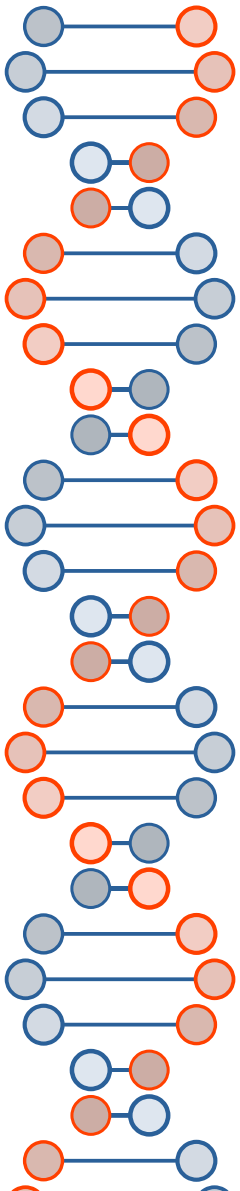
- For the speed testing, I will be using a much larger grid, with more total days but less output frequency to see how much the parallelism can make a difference.
- Grid will be 1000 x 1000.
- Total days will be 100.
- Output frequency will be 50.

Serial_Basic speed



Parallel_Basic Speed Using 1,2,4,8 threads

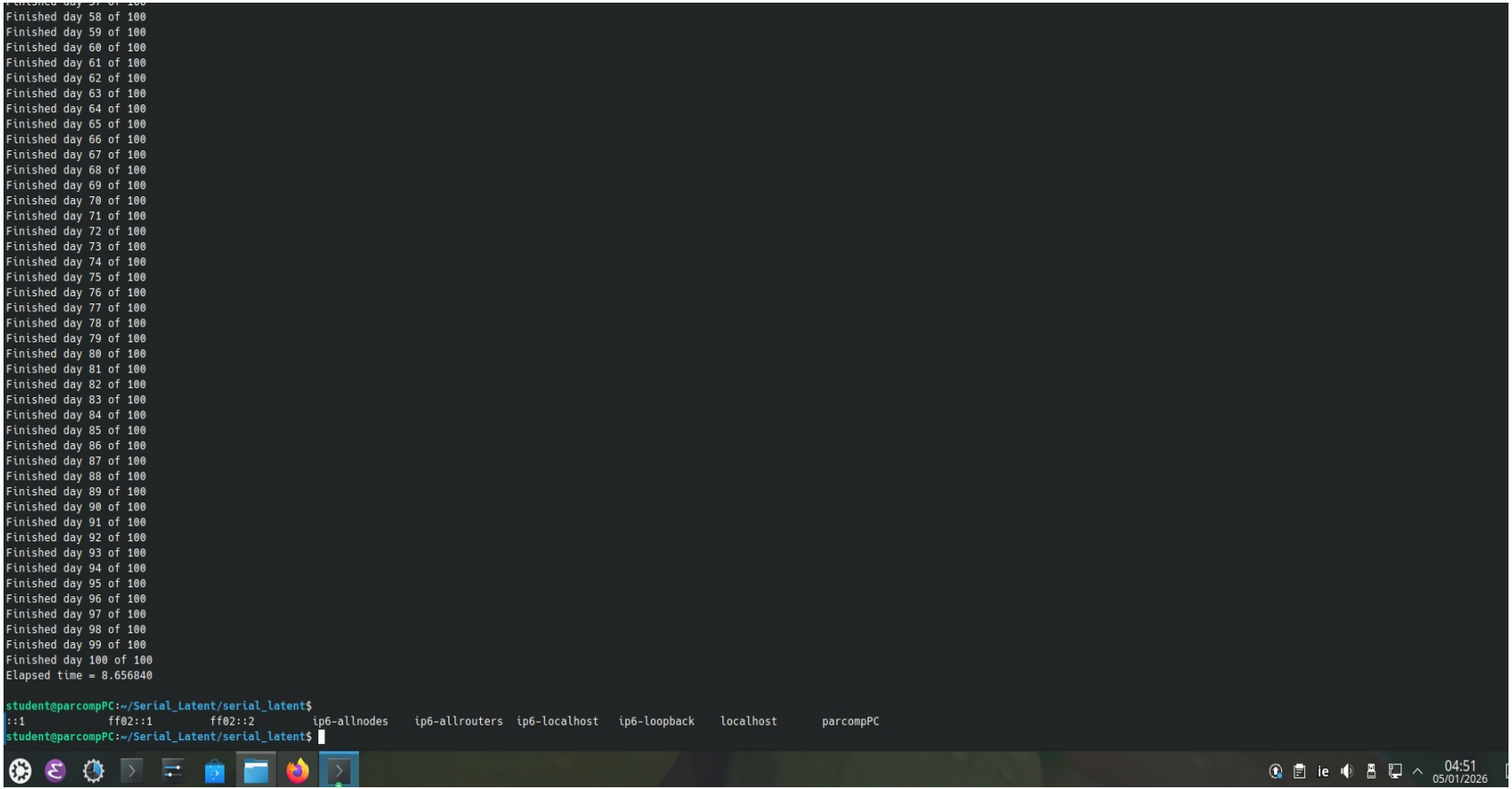




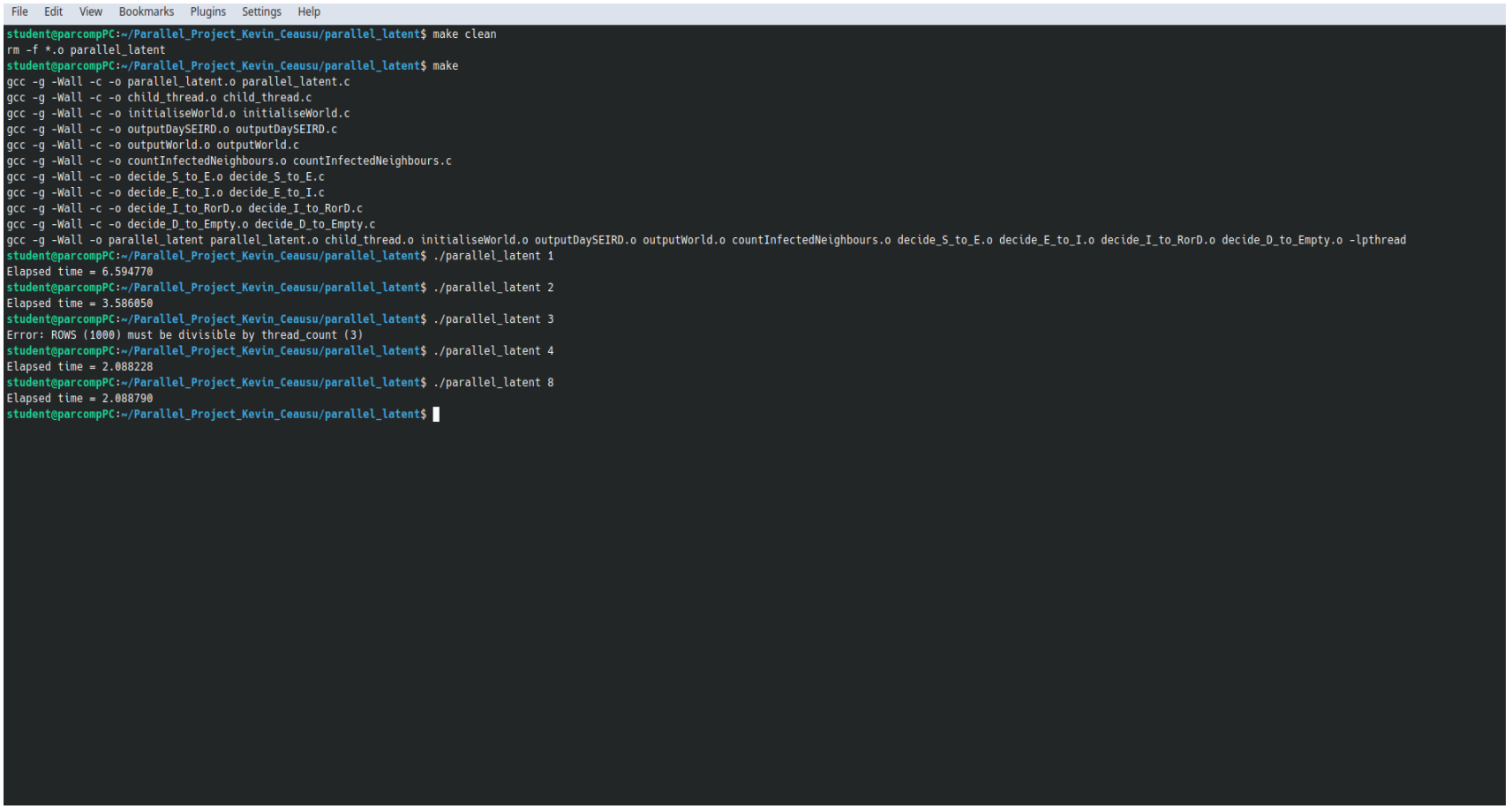
Comparing Serial Basic vs Parallel Basic

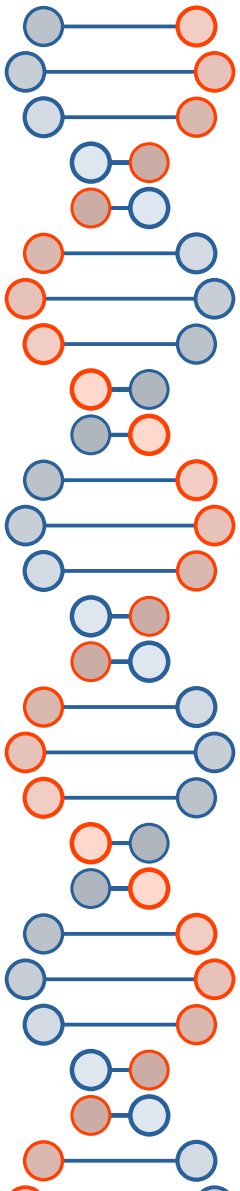
- serial_basic = 6.770045 seconds
- parallel_basic 1 = 6.020715 seconds about 11% faster, about 1.12x faster.
- parallel_basic 2 = 2.918854 seconds about 56.9% faster, about 2.32× faster.
- parallel_basic 4 = 1.747474 seconds about 74.2%, about 3.87x faster.
- parallel_basic 8 = 1.797826 seconds about 73.45%, about 3.77x faster.

Serial_latent speed



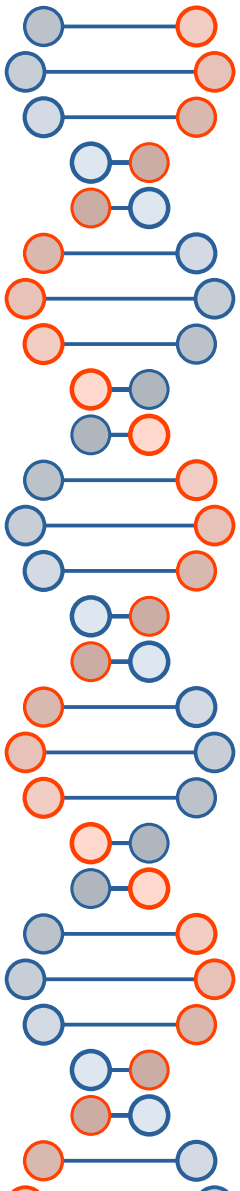
Parallel_latent speed using 1,2,4,8 threads





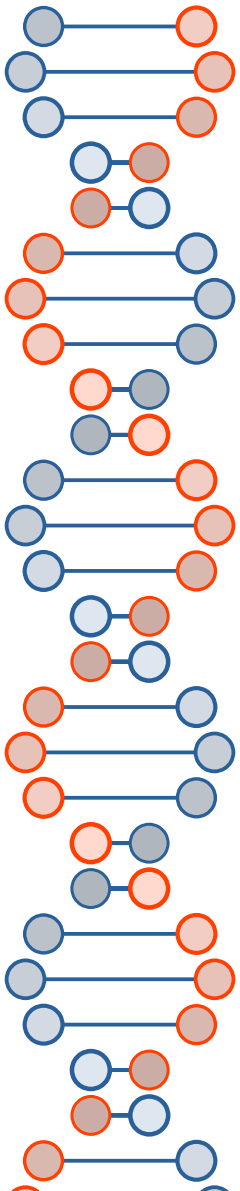
Comparing serial_latent vs Parallel_latent

- serial_latent = 8.656840 seconds
- parallel_latent 1 = 6.594770 seconds, about 24% faster, about 1.31x faster.
- parallel_latent 2 = 3.586050 seconds, about 59% faster, about 2.41x faster.
- parallel_latent 4 = 2.088228 seconds, about 76% faster, about 4.15x faster.
- parallel_latent 8 = 2.088790 seconds, about 76% faster, about 4.14x faster.



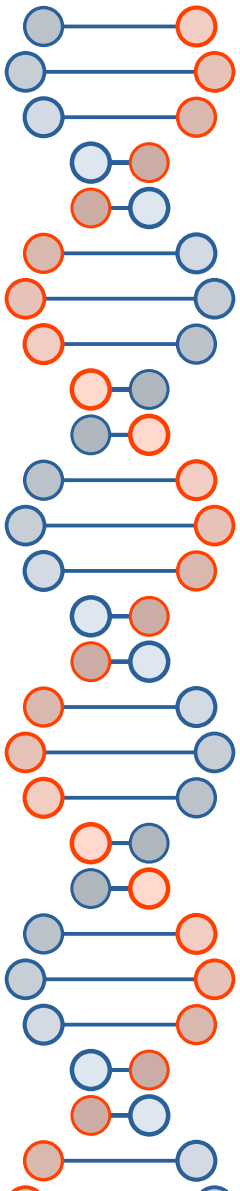
Conclusion of results

- Usually the parallel program using 4 threads gets the best results, I believe that is because I have allocated 4 threads to my Virtual Machine so this is how many it's able to utilise.
- My parallel programs are much faster, for example parallel_latent using 4 threads is 4.15 times faster than serial_latent.
- That is because I lowered the output files frequency as they are not parallel and would slow the speed-up.
- IF I would've ran my programs using let's say output_frequency 1, they wouldn't be this much faster as the serial sections (grid swap, file output) would take most of the time.



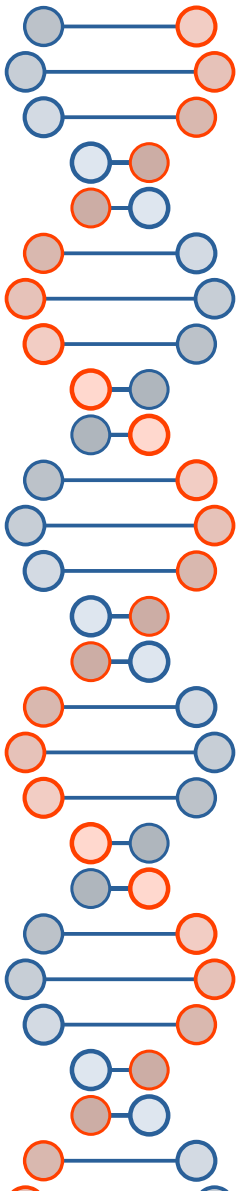
Personal Reflections

- What I liked about the project:
- I liked that the project used a realistic approach. Instead of writing abstract parallel code, I created a real epidemic simulation and could see how different assumptions, such as adding a latent period can change the spread of the virus.
- What I disliked about the project:
- The most challenging aspect of the project was debugging the parallel code. I had instances where the program ran, but the results and outputs were incorrect. It took careful reading and analysing to see where the issues occurred and fix them.



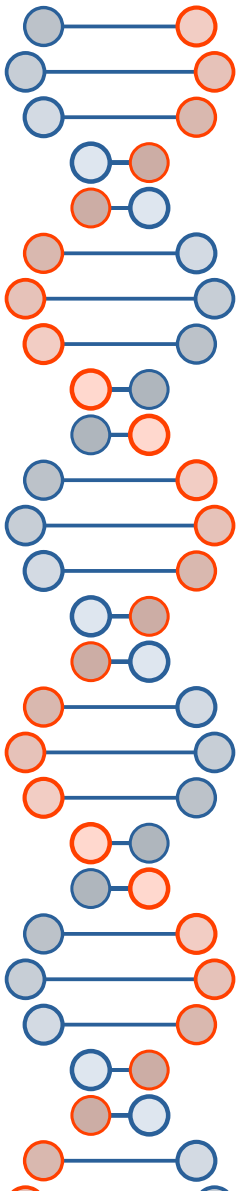
What I would do differently next time

- If I was to start all over again, I would develop the project from the start with parallelism in mind, instead of focusing so much on the serial development, and then adapting it to become parallel.
- Recommendations for the lecturer :
- Overall, I’m very happy with how the lecturer structured this subject for learning, maybe one recommendation would be to introduce students to parallel programming earlier in the labs so they have more time to understand it.



Recommendations for future students

- When building your project, I would advice students to start with a very small grid and low number of days to verify if it works before increasing the size.



Thank you !! :)

by Kevin Ceausu

B00128171